



UNIVERSIDAD
DE MÁLAGA



ESCUELA DE INGENIERÍAS INDUSTRIALES

Departamento: Ingeniería de Sistemas y Automática.

Trabajo de Fin de Máster

Diseño y programación de electrónica de
control para una rueda inteligente.

Máster en Ingeniería Mecatrónica

Autor:

José María Hermoso Ocaña

Tutor:

Carlos Jesús Pérez del Pulgar Mancebo

Málaga, 1 Julio de 2024

DECLARACIÓN EXPLÍCITA DE ORIGINALIDAD

NOMBRE DEL ALUMNO: José María Hermoso Ocaña

TÍTULO DEL TRABAJO: Diseño y programación de electrónica de control para una rueda inteligente.

TUTOR DEL TRABAJO: Carlos Jesús Pérez del Pulgar Mancebo

Declaro que la presente memoria se corresponde con un trabajo plenamente original.

Declaro que he citado debidamente todas las fuentes utilizadas en este trabajo.

Palabras clave:

IMPASS, Robot de Exploración, PCB, ESP32, Control de Motores, MQTT, Node-RED, Raspberry Pi 4, Controlador PID, Sensores de Corriente ACS712, L298, Altium Designer, Protección EMC, Wi-Fi.

Resumen:

Este Trabajo de Fin de Máster se centra en el diseño, desarrollo e implementación de un sistema de control para un robot basado en la plataforma IMPASS (Intelligent Mobility Platform with Active Spoke System). El objetivo principal es maximizar la eficiencia y fiabilidad del sistema electrónico de control de las ruedas inteligentes mediante la integración de una PCB personalizada y el uso del microcontrolador ESP32.

Se ha realizado el diseño de una PCB en el software de Altium Designer aplicando técnicas de protección EMC, capaz de realizar el control de tres motores paso a paso, integrando sensores de corriente para conocer y controlar los consumos de cada uno de estos motores.

El sistema está controlado por un ESP32 el cual gestiona la longitud de los radios de la rueda inteligente para su desplazamiento. Para su funcionamiento se ha implementado técnicas paralelismo, implementación de controladores PID para la gestión del consumo de motores en estado de reposo y comunicación por protocolo MQTT.

Se ha configurado una Raspberry Pi 4 como broker MQTT utilizando Mosquitto y Node-RED, facilitando la comunicación entre los ESP32 y permitiendo un monitoreo en tiempo real de las variables críticas del sistema

En conclusión, este TFM presenta una solución integrada y eficiente para el control de un robot de exploración basado en IMPASS, demostrando mejoras significativas en la precisión, consumo, fiabilidad y escalabilidad del sistema.

Abstract

This Master Thesis focuses on the design, development and implementation of a control system for a robot based on the IMPASS platform (Intelligent Mobility Platform with Active Spoke System). The main objective is to maximise the efficiency and reliability of the electronic control system of the intelligent wheels through the integration of a custom PCB and the use of the ESP32 microcontroller.

A PCB has been designed using Altium Designer software, applying EMC protection techniques, capable of controlling three stepper motors, integrating current sensors to know and control the consumption of each of these motors.

The system is controlled by an ESP32 which manages the length of the spokes of the intelligent wheel for its displacement. Parallelism techniques, implementation of PID controllers for the management of motor consumption in the idle state and communication via MQTT protocol have been implemented for its operation.

A Raspberry Pi 4 has been configured as an MQTT broker using Mosquitto and Node-RED, facilitating communication between the ESP32 and allowing real-time monitoring of critical system variables.

In conclusion, this TFM presents an integrated and efficient solution for the control of an IMPASS-based exploration robot, demonstrating significant improvements in the accuracy, reliability and scalability of the system.

Índice

1	INTRODUCCIÓN	15
1.1	Antecedentes	15
1.2	Objetivos	20
1.3	Motivación	20
2	ESTADO DEL ARTE	23
2.1	Evolución de los Robots de Exploración Planetaria: Sistemas de Movimiento....	24
2.1.1	Primeras Misiones: Lunokhod y Apollo	24
2.1.2	Exploración de Marte: Sojourner, Spirit, Opportunity y Curiosity	25
2.1.3	Avances Recientes y Futuras Misiones	27
2.2	Revisión de modelos previos de IMPASS	28
2.3	Impacto de la Comunicación Wi-Fi de Microcontroladores en la Industria	29
3	DISEÑO DE LA ELECTRÓNICA DE CONTROL	31
3.1	Microcontrolador ESP32.....	31
3.1.1	Características del ESP32 DevKit V1	33
3.1.2	Conexiones del ESP32	34
3.2	Sensores de corriente.....	35
3.2.1	Características del Sensor ACS712	36
3.2.2	Conexiones Sensor ACS712	38
3.3	Motores paso a paso bifásicos	39
3.3.1	Características del Motor Paso a Paso 17HS15-1684S-PG5	40
3.3.2	Conexiones de los Motores Paso a Paso	40
3.4	Puente H L298N	41
3.4.1	Características del L298	41
3.4.2	Ventajas del L298	42
3.4.3	Esquema de Conexión.....	42
3.5	Regulador de tensión.....	44

3.5.1	Características del L7805CV	44
3.5.2	Ventajas del L7805CV	45
3.5.3	Esquema de Conexión.....	45
3.6	Raspberry Pi 4	46
3.6.1	Características de la Raspberry Pi	47
3.6.2	Función en el Proyecto	47
4	DISEÑO HARDWARE.....	49
4.1	Introducción a Altium Designer	49
4.2	Diseño del esquemático	49
4.2.1	Componentes y librerías	49
4.2.2	Creación del Esquemático:	52
4.3	Diseño de la PCB.....	55
4.3.1	Reglas de diseño	55
4.3.2	Dimensiones de la PCB	57
4.3.3	Colocación de componentes en la PCB:	58
4.3.4	Dimensionamiento de pistas.....	60
4.3.5	Enrutamiento de pistas	60
4.3.6	Serigrafía.....	62
4.3.7	Verificación y Simulación	63
4.4	Generación de Archivos de Fabricación	64
4.4.1	BOM	64
4.4.2	Gerber:	66
4.5	Montaje de componentes	67
4.5.1	Montaje de Componentes Surface-Mount (SMD).....	67
4.5.2	Montaje de Componentes Through-Hole (TH)	68
4.5.3	PCB montada.....	69
5	PROGRAMACIÓN FIRMWARE ESP32.....	72
5.1	Introducción.....	72
5.2	Setup y conexión WiFi	74
5.3	Comunicación MQTT	75

5.4	Control de motores paso a paso	77
5.5	Lectura de sensores	79
5.6	Control PID para PWM	80
5.7	Máquina de Estados para el Control de Movimiento.....	82
5.8	Bucles de control	85
5.8.1	Bucle 1: Control de Motores Paso a Paso	85
5.8.2	Bucle 2: Manejo de MQTT y PWM	85
6	PROGRAMACIÓN DE RASPBERRY PI	88
6.1	Introducción al MQTT	88
6.2	Instalación y configuración del Broker MQTT.....	88
6.3	Programación con Node-Red para la gestión de mensajes	89
6.3.1	Conexiones de Velocidad y Posición.....	89
6.3.2	Conexiones de Sensores de Corriente	89
7	RESULTADOS OBTENIDOS	92
8	CONCLUSIONES Y PUNTOS DE MEJORA	94
9	REFERENCIAS.....	96
10	ANEXO A PLANOS PCB.....	98

Índice de Figuras

Figura 1. RoMeLa IMPASS (ROMELA, s.f.).....	16
Figura 2. Prototipo previo IMPASS.	17
Figura 3. Cableado previo.....	18
Figura 4. Cableado previo ruedas I.	19
Figura 5. Cableado previo ruedas II.	19
Figura 6. Shakey primer robot movil inteligente (MDPI, 2022).	23
Figura 7. Lunokhod 1 (1970) MDPI, 2022).....	24
Figura 8. Apollo Lunar Roving Vehicle (1971).	25
Figura 9. Sojourner (1997).....	26
Figura 10. Sojourner (1997).	26
Figura 11. Curiosity (2012).....	27
Figura 12. Perseverance (2020).	27
Figura 13. Prototipo IMPASS	28
Figura 14. Esquema conexión WiFi ESP32.	29
Figura 15. ESP32 Devkit V1.	31
Figura 16. Distribución de pines ESP32 Devkit V1.....	34
Figura 17. Sensor de corriente ACS712ELCTR20AT.....	36
Figura 18. Salida de voltaje vs corriente medida por el sensor.	37
Figura 19. Distribución conexiones sensor ACS712.....	38
Figura 20. Motores paso a paso modelo 17HS15-1684S-PG5.....	39
Figura 21. Puente H L298N.	41
Figura 22. Esquema de conexión L298N.	42
Figura 23. Regulador de tensión L7805CV.	44
Figura 24. Conexiones L7805CV.	45
Figura 25. Raspberry Pi 4.	46
Figura 26. Ejemplo de componente al final de su ciclo de vida.	50
Figura 27. Footprint L298N.	51
Figura 28. Modelo 3D L298N.	51
Figura 29. Bloques terminales 2 posiciones.....	52
Figura 30. Esquema alimentación y controlador de tensión.	52
Figura 31. Bloques terminales de 4 posiciones.....	53
Figura 32. Controladores motores paso a paso.	53
Figura 33. Sensores de corriente.	54

Figura 34. Microcontrolador ESP32.	54
Figura 35. Dimensiones PCB.	57
Figura 36. Primera versión PCB IMPASS.	58
Figura 37. Tercera versión de PCB IMPASS.	59
Figura 38. Versión final de PCB IMPASS.	59
Figura 39. Enrutamiento top.	61
Figura 40. Enrutamiento bottom.	61
Figura 41. Serigrafía de alimentación en PCB.	62
Figura 42. Serigrafía laboratorio en PCB.	63
Figura 43, Desing Rule Check.	63
Figura 44. PCB Top.	66
Figura 45. PCB Bottom.	66
Figura 46. Pistola de calor para soldadura SMD.	68
Figura 47. Cautín para soldadura Through-Hole.	68
Figura 48. PCB montada 1.	69
Figura 49. PCB montada 2.	70
Figura 50. PCB montada con microcontrolador.	70
Figura 51. Diagrama de flujo del Microcontrolador.	73
Figura 52. Señal PWM generada.	81
Figura 53. Node-Red.	90
Figura 54. Panel Node-Red.	91
Figura 55. Cableado previo vs actual.	92
Figura 56. Estado previo vs actual ruedas.	93

Índice de Tablas

Tabla 1. Conexiones Motores paso a paso al ESP32.	35
Tabla 2. Conexiones Sensores ACS712 a ESP32.	35
Tabla 3. Características sensor ACS712	36
Tabla 4. Conexiones sensor ACS712.	38
Tabla 5. Características principales motores paso a paso.	40
Tabla 6. Conexiones motores paso a paso.	40
Tabla 7. Características L298.	41
Tabla 8. Conexiones L298.	43
Tabla 9. Características L7805CV.	44
Tabla 10. Características Raspberry Pi 4.	47
Tabla 11. BOM.....	65

Índice de Códigos

Código 1. setup.....	74
Código 2. setup_wifi	75
Código 3. reconnect.....	76
Código 4. Callbacks.....	77
Código 5. ControlSteps.....	78
Código 6. ReadSensors.....	80
Código 7. ControlPWM.	81
Código 8. Máquina de estados.	85
Código 9. Bucle1.	85
Código 10. Bucle2.	86

1 INTRODUCCIÓN

En el ámbito de la ingeniería mecatrónica, el diseño y la programación de sistemas de control electrónico son elementos fundamentales para el desarrollo de dispositivos innovadores y eficientes. En este contexto, el presente Trabajo de Fin de Máster (TFM) se centra en el diseño y programación de electrónica de control para una rueda inteligente, como parte de una serie de proyectos previos que han abordado la fabricación y control de un prototipo de robot basado en la plataforma IMPASS (Intelligent Mobility Platform for Autonomous Systems and Space).

El proyecto en cuestión se distingue por su enfoque en el rediseño del circuito electrónico de control, su implementación en una PCB de dimensiones específicas y la implementación de un protocolo de comunicación WiFi entre las distintas partes que conforman el robot móvil. La adopción de una PCB (Placa de Circuito Impreso) como plataforma para el control electrónico reviste una importancia significativa en la industria actual debido a sus múltiples ventajas. Las PCBs proporcionan un medio eficiente y confiable para interconectar componentes electrónicos, optimizando la disposición y el enrutamiento de las pistas conductoras. Además, permiten una integración compacta de los componentes, lo que reduce el espacio ocupado y simplifica el ensamblaje y la producción en masa.

A lo largo de las siguientes secciones, se detallará el proceso seguido para el diseño y la implementación de la electrónica de control en la PCB, el diseño del firmware de control, así como los resultados obtenidos y antecedentes.

1.1 ANTECEDENTES

El desarrollo de robots móviles de exploración planetaria ha sido objeto de investigación y desarrollo durante décadas, impulsado por la búsqueda de comprender mejor el universo que nos rodea y explorar entornos hostiles más allá de nuestro planeta. Desde los primeros intentos pioneros hasta las misiones más recientes, la evolución de estos robots ha sido marcada por avances significativos en diseño, autonomía, capacidades de detección y manipulación, así como en la robustez y adaptabilidad de sus sistemas.

Los primeros prototipos de IMPASS fueron desarrollados a mediados de la década de 2000 por el Robotics and Mechanisms Laboratory (RoMeLa) en Virginia Tech (ROMELA, s.f.). El diseño inicial se centró en un sistema de ruedas con radios activos que podían cambiar de longitud para adaptarse a diferentes tipos de terreno, proporcionando así una mayor flexibilidad y capacidad de movimiento en comparación con los sistemas de locomoción tradicionales. Estos primeros modelos demostraron la viabilidad del concepto de ruedas con radios activos, aunque enfrentaron desafíos en términos de control y robustez mecánica.

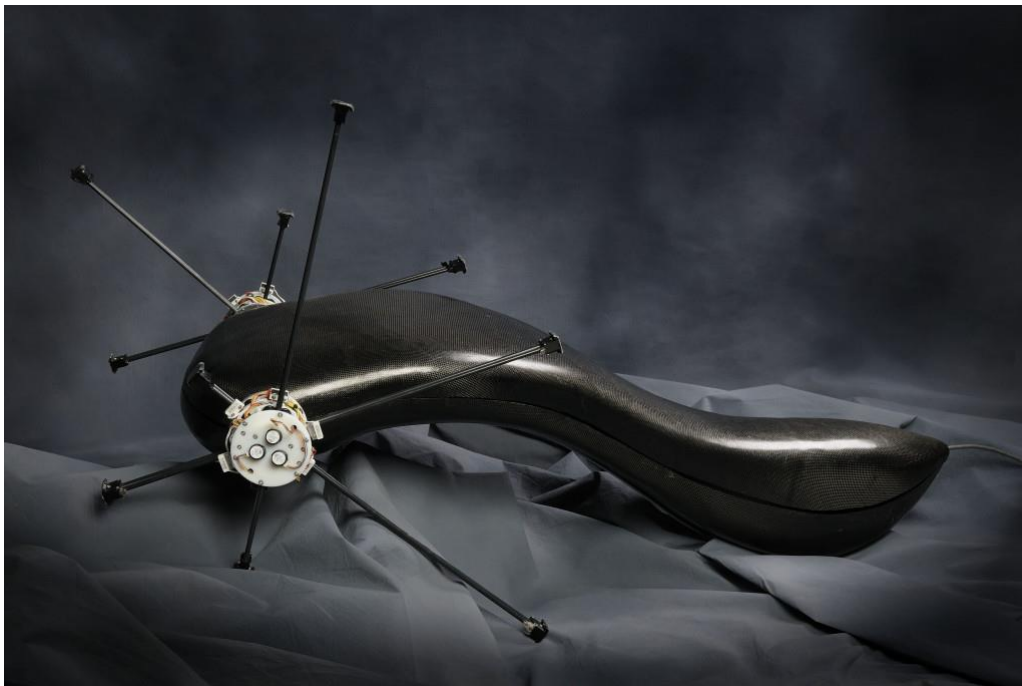


Figura 1. RoMeLa IMPASS (ROMELA, s.f.)

A lo largo de los años, el diseño de IMPASS ha evolucionado significativamente. Las versiones posteriores han incorporado mejoras en la precisión de los sensores, la capacidad de procesamiento y la integración de algoritmos de control más avanzados. Estas mejoras han permitido a los robots IMPASS operar de manera más eficiente en una variedad de entornos, incluyendo terrenos rocosos y accidentados. La evolución continua de esta tecnología ha llevado a un aumento en la autonomía y la capacidad de adaptarse a diferentes condiciones del terreno, haciendo que los robots IMPASS sean cada vez más útiles para misiones de exploración planetaria y otras aplicaciones en entornos hostiles.

Un antecedente importante de este trabajo es el TFM titulado "Fabricación y control del movimiento de un prototipo basado en IMPASS" realizado por Antonio López Palomeque. En dicho proyecto, se abordó el desarrollo de un prototipo de robot basado

en la plataforma IMPASS, destacando la integración de diversos componentes electrónicos y mecánicos para lograr un control preciso del movimiento. El enfoque principal se centró en la construcción y control de la estructura física del robot, utilizando un único microcontrolador para la gestión de las ruedas y la navegación del robot.

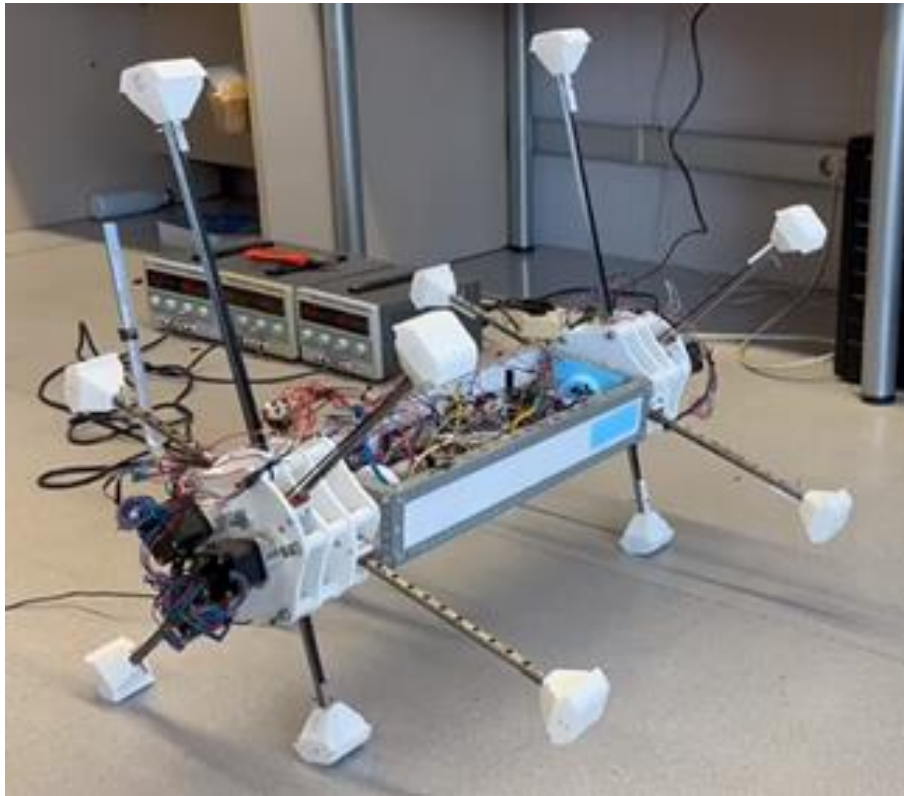


Figura 2. Prototipo previo IMPASS.

El proyecto mencionado anteriormente, llevado a cabo en el laboratorio de robótica espacial de la universidad de Málaga, sentó las bases para la creación de un sistema de control eficiente y robusto, utilizando microcontroladores para la gestión de las ruedas y la navegación del robot. A pesar de los avances logrados, surgieron desafíos significativos relacionados con la comunicación entre los componentes del sistema y la complejidad del cableado necesario para su funcionamiento, lo que motivó la necesidad de buscar soluciones más optimizadas y efectivas.

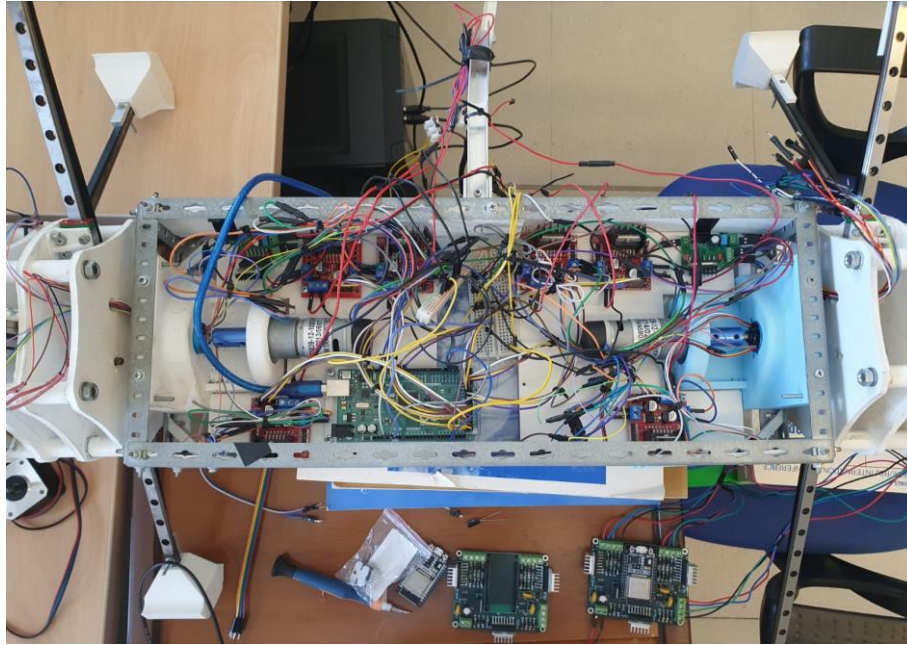


Figura 3. Cableado previo.

Como se puede observar en la figura 3, el cableado de la electrónica de control del IMPASS supone un problema a la hora de su fiabilidad debido a que una posible desconexión supondría un mal funcionamiento de todo el sistema o incluso podría suponer daños en la electrónica del circuito.

En cuanto al cableado de los motores paso a paso situados en las ruedas, véase figuras 4 y 5, supone un problema la cantidad cables que es necesario pasar al cuerpo del robot y como estos están estos situados.

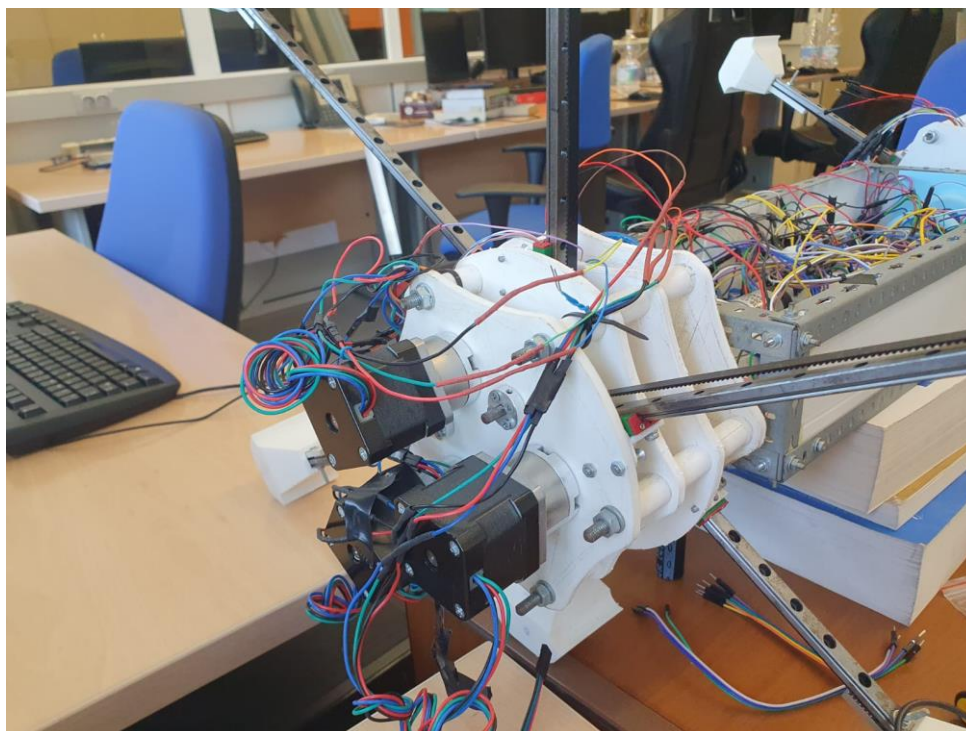


Figura 4. Cableado previo ruedas I.

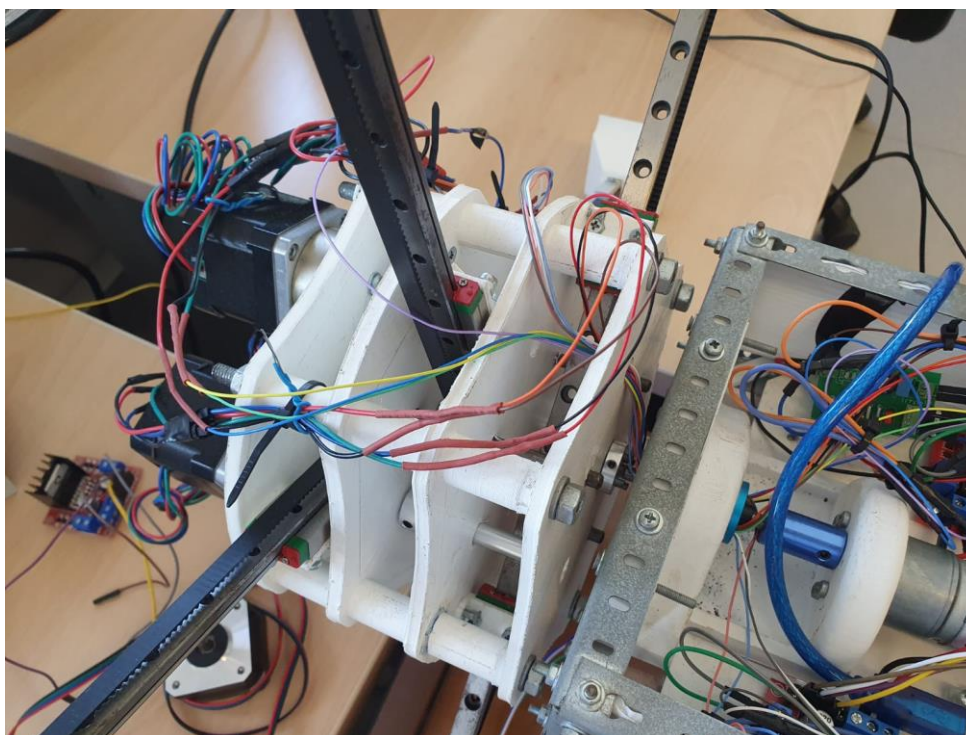


Figura 5. Cableado previo ruedas II.

1.2 OBJETIVOS

La decisión de diseñar y fabricar una PCB específica para este proyecto se fundamenta en la necesidad de maximizar la eficiencia y la fiabilidad del sistema de control electrónico de la rueda inteligente. Además, esta implementación permite reducir el espacio necesario para el control de las ruedas, dejando más espacio disponible para futuras mejoras. Los objetivos específicos de este TFM incluyen:

- **Realizar un estudio** exhaustivo del estado y comportamiento actual del IMPASS para comprender en detalle su funcionamiento y los puntos de mejora.
- **Diseñar la electrónica de control** necesaria para la rueda inteligente y su integración en una PCB.
- **Diseñar una PCB** conforme a las limitaciones de dimensiones máximas establecidas por el diseño de la rueda.
- **Programar el microcontrolador ESP32** para controlar las ruedas inteligentes y establecer un protocolo de comunicación con el ordenador de a bordo Raspberry Pi.
- **Desarrollar un protocolo de comunicación efectivo** entre el ESP32 y la Raspberry Pi para la transmisión de datos necesarios mediante Wi-Fi, facilitando tanto el control de la rueda como la recepción de datos de sensores conectados al ESP32.
- **Implementación de sensores de corriente** para realizar un control de bucle cerrado para el control de los motores.

1.3 MOTIVACIÓN

La motivación detrás de este proyecto radica en la necesidad de mejorar la comunicación entre las distintas partes del robot y reducir el cableado necesario, lo que se traduce en una mayor fiabilidad y facilidad de mantenimiento del sistema. La capacidad de adaptar dinámicamente la longitud de los radios de la rueda para enfrentar diversos tipos de terreno gestionada por un microcontrolador ESP32 representa un desafío técnico y científico que involucra la aplicación de conocimientos interdisciplinarios en áreas como la ingeniería electrónica, la mecánica y la informática.

Además, la implementación de una PCB específica permitirá una integración más compacta y eficiente de los componentes electrónicos, optimizando el espacio y dando lugar a la capacidad de implementar nuevas mejoras en el futuro.

La adopción de tecnologías avanzadas de comunicación, como el Wi-Fi, permite una interacción más flexible y remota con el sistema, abriendo nuevas posibilidades para el control y monitoreo del robot en tiempo real. Este proyecto, por lo tanto, no solo busca resolver problemas técnicos específicos, sino también contribuir al avance de la ingeniería mecatrónica mediante el desarrollo de soluciones innovadoras y aplicables a una amplia gama de aplicaciones industriales y de investigación.

2 ESTADO DEL ARTE

El desarrollo de robots móviles de exploración planetaria ha sido objeto de investigación y desarrollo durante décadas, impulsado por la búsqueda de comprender mejor el universo que nos rodea y explorar entornos hostiles más allá de nuestro planeta. Desde los primeros intentos pioneros hasta las misiones más recientes, la evolución de estos robots ha sido marcada por avances significativos en diseño, autonomía, capacidades de detección y manipulación, así como en la robustez y adaptabilidad de sus sistemas de control.

Un ejemplo clave de estos avances es Shakey, el primer robot móvil inteligente desarrollado en la década de 1960 por el Stanford Research Institute (SRI). Shakey fue pionero en la integración de percepción, razonamiento y acción, utilizando un sistema de cámara y un conjunto de algoritmos que le permitían navegar y realizar tareas en un entorno controlado (The World's First Mobile Intelligent Robot, 1972) (MDPI, 2022)

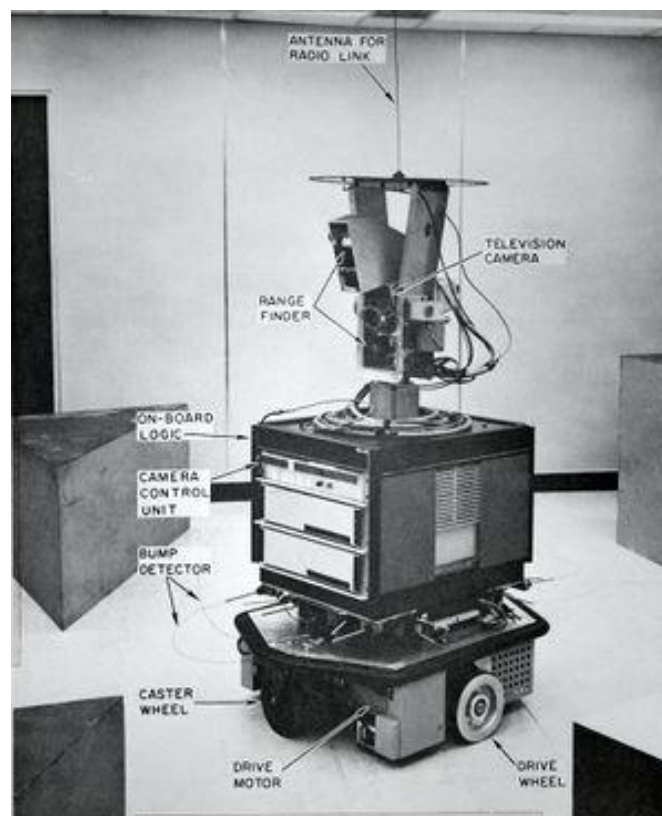


Figura 6. Shakey primer robot movil inteligente (MDPI, 2022).

2.1 EVOLUCIÓN DE LOS ROBOTS DE EXPLORACIÓN PLANETARIA: SISTEMAS DE MOVIMIENTO

La exploración planetaria ha sido una de las áreas más fascinantes y tecnológicamente desafiantes de la ingeniería espacial. Los robots móviles de exploración planetaria, como los Rover enviados a Marte por la NASA, tales como el Sojourner, Opportunity y Curiosity, han sido diseñados para operar en entornos extremadamente desafiantes, incluyendo terrenos rocosos, pendientes pronunciadas y variaciones extremas de temperatura. Estos robots han demostrado la viabilidad y la importancia de la exploración robótica en la investigación espacial, proporcionando datos cruciales sobre la geología, la atmósfera y la posible habitabilidad de otros planetas

2.1.1 Primeras Misiones: Lunokhod y Apollo

Los primeros robots de exploración lunar, como el Lunokhod 1 de la Unión Soviética y el Rover de la misión Apollo de Estados Unidos, sentaron las bases para el desarrollo de sistemas de movimiento en entornos extraterrestres.

Lunokhod 1 (1970): Fue el primer Rover robótico controlado remotamente que aterrizó en la Luna. Utilizaba un sistema de locomoción basado en ocho ruedas independientes con suspensión, permitiendo una distribución uniforme del peso y mejor capacidad de manejo en terrenos irregulares (Stanford University, 2019).

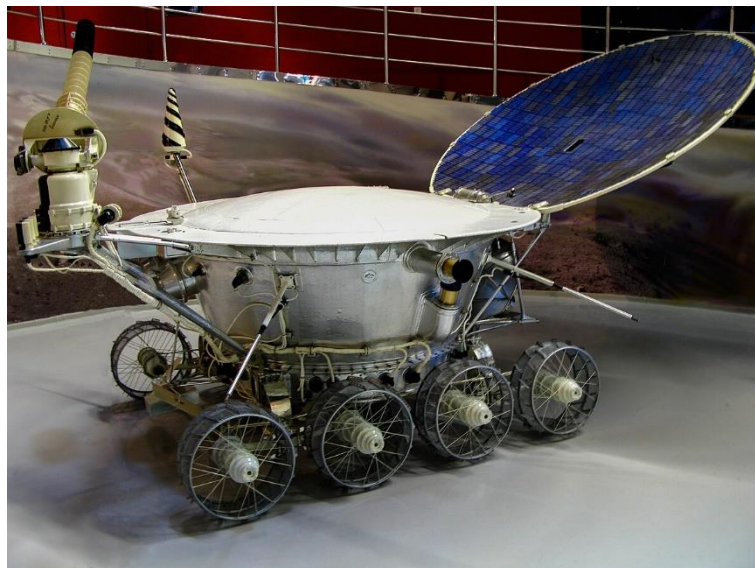


Figura 7. Lunokhod 1 (1970) MDPI, 2022).

- **Apollo Lunar Roving Vehicle (1971):** Utilizado durante las misiones Apollo 15, 16 y 17, este Rover tenía cuatro ruedas motrices, cada una con su propio motor eléctrico. Su diseño incluyó neumáticos de malla metálica con tacos de titanio para mejorar la tracción en la superficie lunar (Design and Research of All-Terrain Wheel-Legged Robot, 2021).

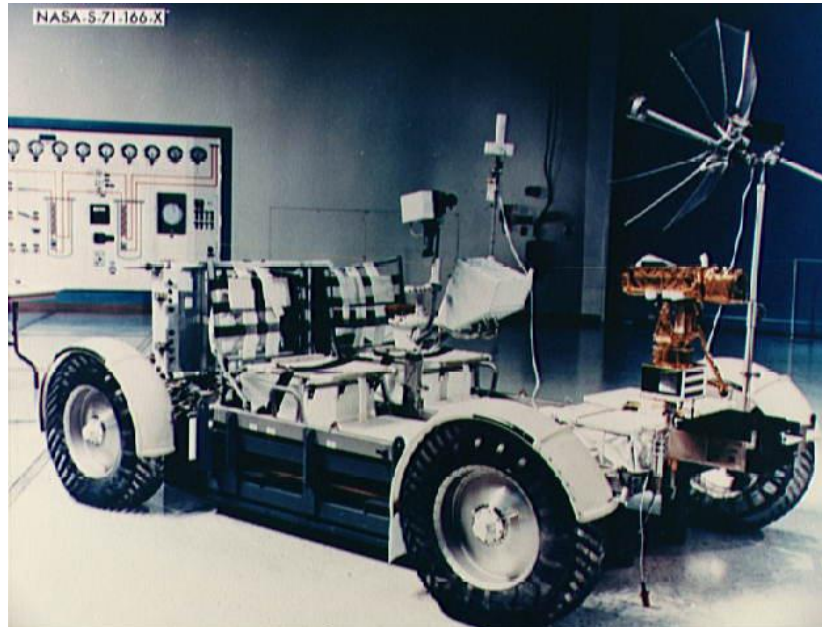


Figura 8. Apollo Lunar Roving Vehicle (1971).

2.1.2 Exploración de Marte: Sojourner, Spirit, Opportunity y Curiosity

La exploración de Marte presentó nuevos desafíos debido a la complejidad y variedad del terreno marciano. La NASA ha liderado varios proyectos exitosos que han demostrado avances notables en los sistemas de movimiento de los Rover.

- **Sojourner (1997):** Fue el primer Rover de la NASA en Marte, parte de la misión Mars Pathfinder. Utilizaba un sistema de seis ruedas con suspensión tipo rocker-bogie, lo que le permitía superar obstáculos del doble del diámetro de sus ruedas sin volcarse (NASA, 1996)



Figura 9. Sojourner (1997).

- **Spirit y Opportunity (2004):** Estos rovers gemelos utilizaron también el sistema de suspensión rocker-bogie. Esta configuración de seis ruedas proporcionó una mayor estabilidad y capacidad para sortear obstáculos, permitiendo a los rovers cubrir distancias significativas y explorar varios cráteres marcianos (NASA, 2004) .

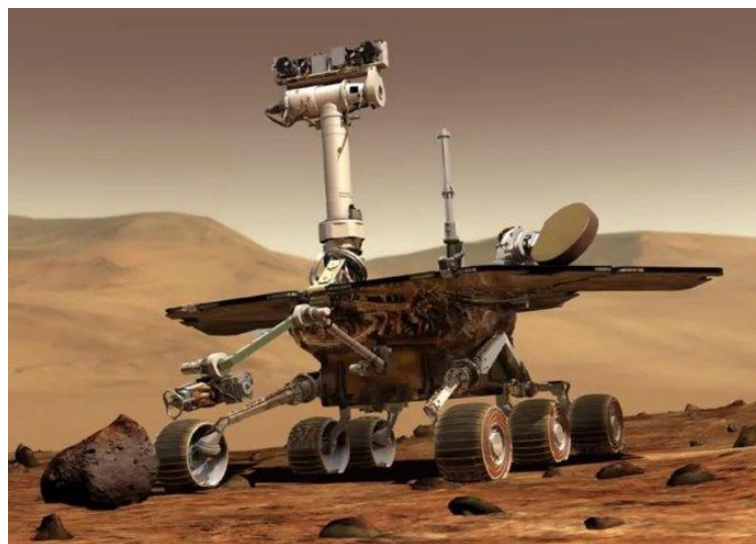


Figura 10. Sojourner (1997).

- **Curiosity (2012):** Curiosity continuó con el diseño de suspensión rocker-bogie pero incluyó mejoras significativas en la durabilidad de las ruedas y la capacidad de navegación autónoma. Equipado con un sistema de propulsión nuclear, ha

sido capaz de realizar una exploración prolongada y más profunda de la superficie de Marte (NASA, 2012).



Figura 11. Curiosity (2012).

2.1.3 Avances Recientes y Futuras Misiones

Los avances tecnológicos continúan mejorando las capacidades de los rovers de exploración planetaria, centrándose en sistemas de movimiento más avanzados y adaptables.

- **Perseverance (2020):** Siguiendo el legado de Curiosity, Perseverance utiliza una versión mejorada del sistema rocker-bogie. Sus ruedas han sido rediseñadas para mayor resistencia a daños, y el rover incluye un helicóptero, Ingenuity, que representa un avance en la exploración aérea en Marte (NASA, s.f.).



Figura 12. Perseverance (2020).

- **Rovers de Próxima Generación:** Las futuras misiones planean utilizar sistemas de locomoción más avanzados, como ruedas deformables, patas robóticas y sistemas híbridos que combinan ruedas y patas para mayor versatilidad y capacidad de adaptación a terrenos extremadamente accidentados. Estos sistemas buscan emular la flexibilidad y la capacidad de adaptación que se observa en los robots desarrollados por ROMELA y otras instituciones de investigación robótica

2.2 REVISIÓN DE MODELOS PREVIOS DE IMPASS

En este contexto, los modelos previos del robot IMPASS representan un paso importante en la evolución de los robots móviles de exploración planetaria. El proyecto IMPASS (Intelligent Mobility Platform for Autonomous Systems and Space) ha sido desarrollado por diversas instituciones y equipos de investigación, con el objetivo de crear un robot móvil autónomo capaz de operar en entornos hostiles y variados, incluyendo terrenos lunares y marcianos simulados en la Tierra.

Los modelos previos del robot IMPASS han incorporado diversas innovaciones en diseño mecánico, sistemas de locomoción, sensores de navegación y control de movimiento. Estos robots han sido utilizados en numerosas pruebas y experimentos en condiciones controladas y semi-reales, con el fin de validar su rendimiento y fiabilidad en entornos similares a los planetas y lunas del sistema solar.

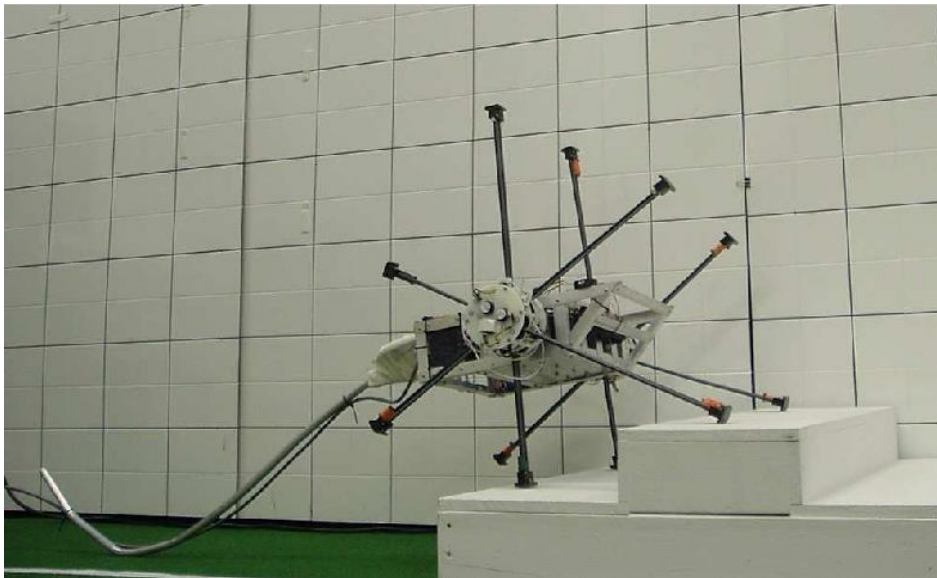


Figura 13. Prototipo IMPASS

IMPASS utiliza un sistema de radios activos que permite que las ruedas cambien de forma y tamaño para superar obstáculos y adaptarse a terrenos variados. Este

sistema se diferencia de los sistemas de locomoción tradicionales y proporciona una mayor flexibilidad y capacidad de adaptación. Según la descripción del proyecto en la página de ROMELA, los modelos IMPASS (véase figura 1) ha demostrado su eficacia en terrenos desafiantes, combinando las ventajas de las ruedas y las patas para mejorar la movilidad del robot.

La experiencia acumulada a lo largo de los proyectos previos de IMPASS ha proporcionado importantes lecciones y conocimientos sobre los desafíos técnicos y operativos asociados con la exploración robótica en entornos extraterrestres. Los avances en el diseño y la programación de sistemas de control electrónico, así como en la integración de sensores y actuadores, han allanado el camino para el desarrollo de soluciones más avanzadas y eficientes, como la rueda inteligente objeto de estudio en este trabajo.

2.3 IMPACTO DE LA COMUNICACIÓN WI-FI DE MICROCONTROLADORES EN LA INDUSTRIA

La evolución de la tecnología de comunicación inalámbrica ha transformado profundamente la manera en que interactuamos con los dispositivos electrónicos en todos los ámbitos de la vida cotidiana. En particular, la proliferación de microcontroladores equipados con capacidades de comunicación Wi-Fi, como el ESP32, ha tenido un impacto significativo en la industria, abriendo nuevas posibilidades en términos de conectividad, control remoto y monitoreo en tiempo real.

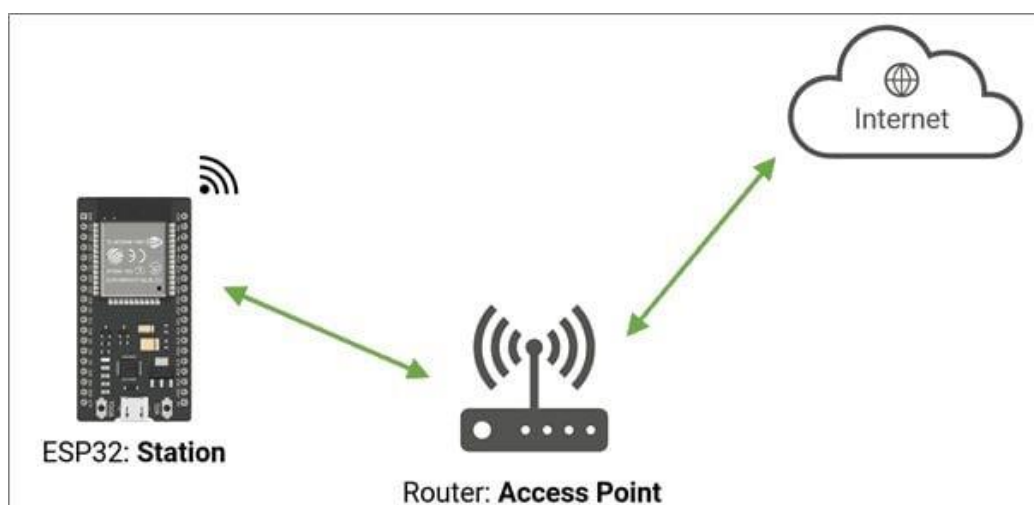


Figura 14. Esquema conexión WiFi ESP32.

La capacidad de los microcontroladores Wi-Fi para conectarse a redes inalámbricas locales y a Internet ha revolucionado los procesos de fabricación, automatización industrial, monitorización de sistemas y control de dispositivos en tiempo real. En entornos industriales, la integración de microcontroladores Wi-Fi en equipos y maquinaria permite la creación de sistemas de control distribuido y la implementación de soluciones de Industria 4.0, facilitando la comunicación entre dispositivos, la recopilación de datos en tiempo real y la optimización de procesos de producción.

Además, la comunicación Wi-Fi de microcontroladores como el ESP32 ha allanado el camino para el desarrollo de dispositivos IoT (Internet de las cosas) en la industria, permitiendo la conexión y la interacción entre una amplia variedad de dispositivos inteligentes y sensores distribuidos en entornos industriales. Esta conectividad inalámbrica posibilita la creación de sistemas de monitorización y control remoto, la implementación de mantenimiento predictivo y la optimización de la eficiencia operativa en plantas industriales y cadenas de producción.

3 DISEÑO DE LA ELECTRÓNICA DE CONTROL

En este capítulo se describirá los principales componentes que forman el diseño de la electrónica de control de la rueda inteligente. La electrónica de control no solo permite la gestión precisa del movimiento y las funciones del robot, sino que también asegura la integración eficiente y fiable de todos los componentes electrónicos involucrados.

En este apartado, se abordarán los diversos aspectos del diseño de la electrónica de control, comenzando con la selección de componentes clave como el microcontrolador ESP32, los controladores de motores y los sensores necesarios para la operación del sistema.

Este enfoque integral garantizará que la electrónica de control diseñada sea robusta, eficiente y capaz de soportar las exigencias operativas del entorno en el que se desplegará la rueda inteligente.

3.1 MICROCONTROLADOR ESP32

El proyecto inicialmente utilizaba el Arduino Mega para el control total del sistema, un microcontrolador conocido por su facilidad de uso y su amplia adopción en proyectos de hobby y prototipos. Sin embargo, a medida que las necesidades del proyecto evolucionaron, se identificaron varias limitaciones en el Arduino Mega que llevaron a la transición al ESP32:

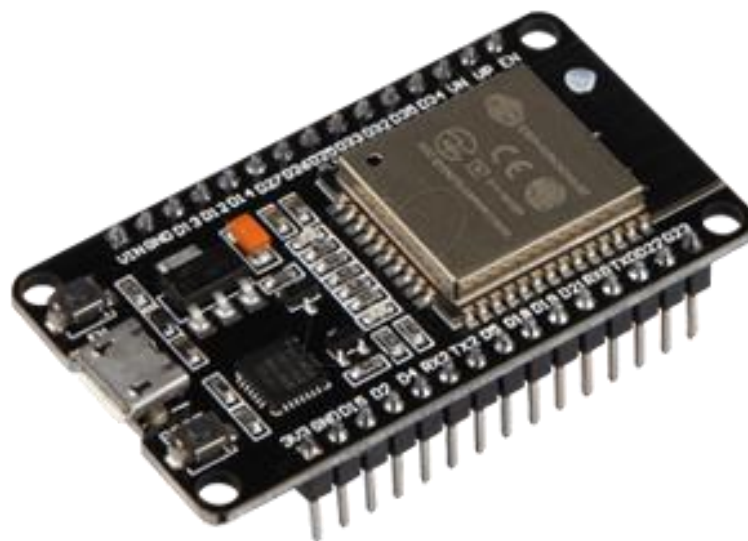


Figura 15. ESP32 Devkit V1.

Conectividad Inalámbrica:

- **Arduino Mega:** No tiene conectividad Wi-Fi o Bluetooth integrada, lo que requiere módulos adicionales y una complejidad incrementada para implementar la comunicación inalámbrica.
- **ESP32:** Incluye Wi-Fi y Bluetooth integrados, simplificando el diseño y reduciendo el costo y el espacio necesario para la conectividad.

Rendimiento del Procesador:

- **Arduino Mega:** Utiliza un microcontrolador AVR de 8 bits con una frecuencia de reloj de 16 MHz, lo que limita su capacidad de procesamiento para aplicaciones intensivas en datos.
- **ESP32:** Equipado con un procesador dual-core de 32 bits que opera a 240 MHz, proporcionando un rendimiento significativamente superior para tareas complejas y multitarea.

Consumo de Energía:

- **Arduino Mega:** Consume más energía, lo que no es ideal para aplicaciones que requieren eficiencia energética o funcionamiento prolongado con baterías.
- **ESP32:** Ofrece modos de bajo consumo de energía, incluyendo Deep Sleep, que son esenciales para aplicaciones de bajo consumo y operaciones prolongadas con baterías.

Capacidades de Periféricos:

- **Arduino Mega:** Aunque tiene una buena cantidad de pines GPIO y periféricos, no puede igualar la versatilidad y la cantidad de periféricos avanzados que ofrece el ESP32.
- **ESP32:** Proporciona una mayor cantidad de ADCs de alta resolución, DACs, canales PWM y opciones de comunicación, lo que permite una mayor flexibilidad en el diseño del sistema.

La transición del Arduino Mega al ESP32 fue impulsada por la necesidad de mayor rendimiento, eficiencia energética y conectividad inalámbrica integrada, lo que permite desarrollar un sistema de control más avanzado y eficiente para la rueda inteligente en este proyecto. Además, esto permite la implementación de un microcontrolador para

cada rueda, dado la posibilidad de implementar la electrónica de control en la misma rueda y reducir el número de cables que es necesario pasar por el eje de la rueda.

El ESP32 DevKit V1 es un microcontrolador desarrollado por Espressif Systems, conocido por su alta integración, capacidades de conectividad y eficiencia energética. A continuación, se presentan las características más destacadas del ESP32 DevKit.

3.1.1 Características del ESP32 DevKit V1

Procesador:

- Dual-core Tensilica LX6, operando a una velocidad de hasta 240 MHz.
- Arquitectura de 32 bits, lo que permite un procesamiento rápido y eficiente.

Memoria:

- 520 KB de SRAM.
- 4 MB de memoria flash externa.

Conectividad:

- Wi-Fi 802.11 b/g/n.
- Bluetooth 4.2 y BLE (Bluetooth Low Energy).

Periféricos Integrados:

- 34 pines GPIO (General Purpose Input/Output).
- 12 ADCs (Analog to Digital Converters) de 12 bits.
- 2 DACs (Digital to Analog Converters).
- 16 canales PWM (Pulse Width Modulation).
- Interfaces de comunicación: SPI, I2C, UART, I2S.

Consumo de Energía:

- Modo de bajo consumo de energía (Deep Sleep) con consumo de corriente de menos de 5 μ A.
- Varios modos de gestión de energía para optimizar el consumo según las necesidades de la aplicación.

3.1.2 Conexiones del ESP32

A continuación, se presenta una breve explicación de las conexiones realizadas en el esquema del circuito, así como tablas detalladas de los pines GPIO utilizados para cada uno de los componentes.

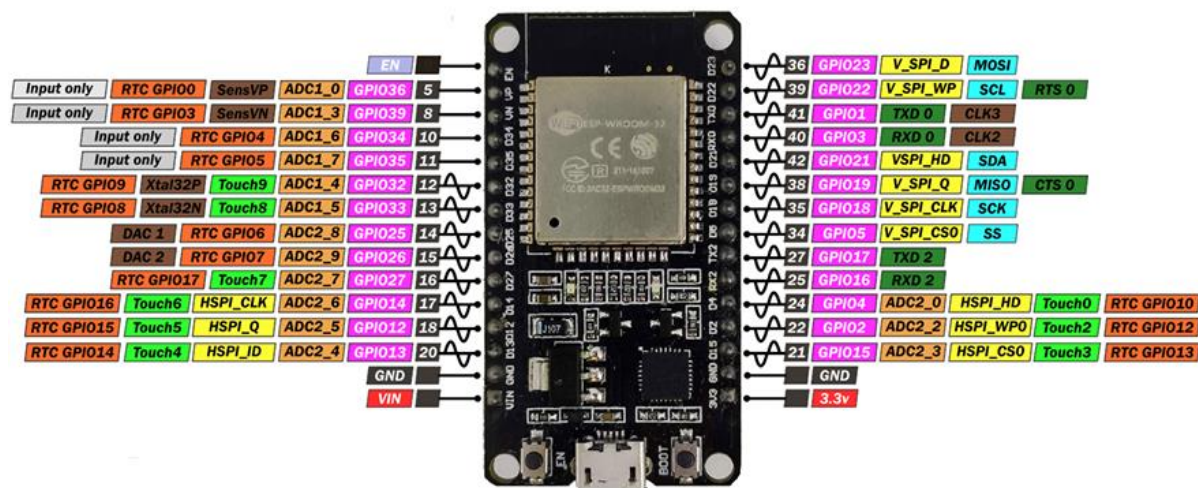


Figura 16. Distribución de pines ESP32 Devkit V1.

Conexiones de los Motores Paso a Paso

Los motores paso a paso están conectados al ESP32 a través de varios pines GPIO, permitiendo el control preciso de cada fase del motor. Cada motor paso a paso requiere cuatro pines de control para manejar sus bobinas. En tabla se muestran las conexiones realizadas:

Motor	Pin del Motor	GPIO del ESP32
Motor 1	IN1	GPIO 26
	IN2	GPIO 25
	IN3	GPIO 33
	IN4	GPIO 32
Motor 2	IN1	GPIO 14
	IN2	GPIO 27
	IN3	GPIO 12

	IN4	GPIO 13
Motor 3	IN1	GPIO 21
	IN2	GPIO 22
	IN3	GPIO 19
	IN4	GPIO 23

Tabla 1. Conexiones Motores paso a paso al ESP32.

Conexiones de los Sensores de Corriente ACS712

Los sensores de corriente ACS712 están conectados a los pines ADC del ESP32 para medir la corriente consumida por cada motor paso a paso. Los valores analógicos leídos de estos sensores son utilizados para monitorear y ajustar el consumo de corriente, asegurando así el funcionamiento eficiente del sistema.

Sensor de Corriente	GPIO del ESP32
ACS712 - Motor 1	GPIO 35
ACS712 - Motor 2	GPIO 21
ACS712 - Motor 3	GPIO 34

Tabla 2. Conexiones Sensores ACS712 a ESP32.

3.2 SENSORES DE CORRIENTE

El sensor de corriente ACS712 es el encargado de medir la corriente que consumen los motores paso a paso. Este sensor, desarrollado por Allegro Microsystems, es ampliamente utilizado en aplicaciones donde se requiere una medición precisa y lineal de la corriente.

La principal función de estos sensores es controlar la corriente consumida por cada uno de los motores. Para ello, se utiliza una señal PWM generada por el microcontrolador para gestionar la corriente, especialmente cuando el motor paso a paso se encuentra en reposo. Esto permite actuar como freno, evitando que los motores cedan debido al peso del robot cuando los motores no están en movimiento.



Figura 17. Sensor de corriente ACS712ELCTR20AT

3.2.1 Características del Sensor ACS712

Característica	Descripción
Rango de Medición	-20A a 20A
Precisión y Linealidad	Salida analógica proporcional a la corriente medida
Sensibilidad	100 mV/A (para la versión ACS712-20B)
Voltaje de Alimentación	5V
Temperatura de Operación	-40°C a 85°C

Tabla 3. Características sensor ACS712

1. Integración Fácil:

- El ACS712 es fácil de integrar en el diseño de la PCB debido a su tamaño compacto y a su salida analógica directa, que puede ser fácilmente leída por un microcontrolador como el ESP32.

2. Aislamiento Galvánico:

- Proporciona aislamiento galvánico entre el circuito de medición de corriente y el circuito de salida, lo que mejora la seguridad y la integridad del sistema.

3. Compatibilidad con Microcontroladores Modernos:

- La señal de salida del ACS712 es compatible con los ADCs (Convertidores Analógico-Digital) integrados en microcontroladores modernos como el ESP32, permitiendo una fácil digitalización de la señal.

4. Costo y Disponibilidad:

- El sensor ACS712 es una solución económica y está ampliamente disponible en el mercado, lo que facilita su adquisición y reemplazo.

El sensor de corriente ACS712 utiliza el efecto Hall para medir la corriente que pasa a través de un conductor. Este efecto es un fenómeno en el que un voltaje (voltaje Hall) es generado perpendicularmente a la dirección del flujo de corriente cuando esta pasa a través de un semiconductor en presencia de un campo magnético. El ACS712 mide este voltaje Hall para determinar la cantidad de corriente que fluye a través del conductor.

El ACS712 ofrece una salida lineal en el rango de corriente especificado. La linealidad asegura que el voltaje de salida sea proporcional a la corriente medida.

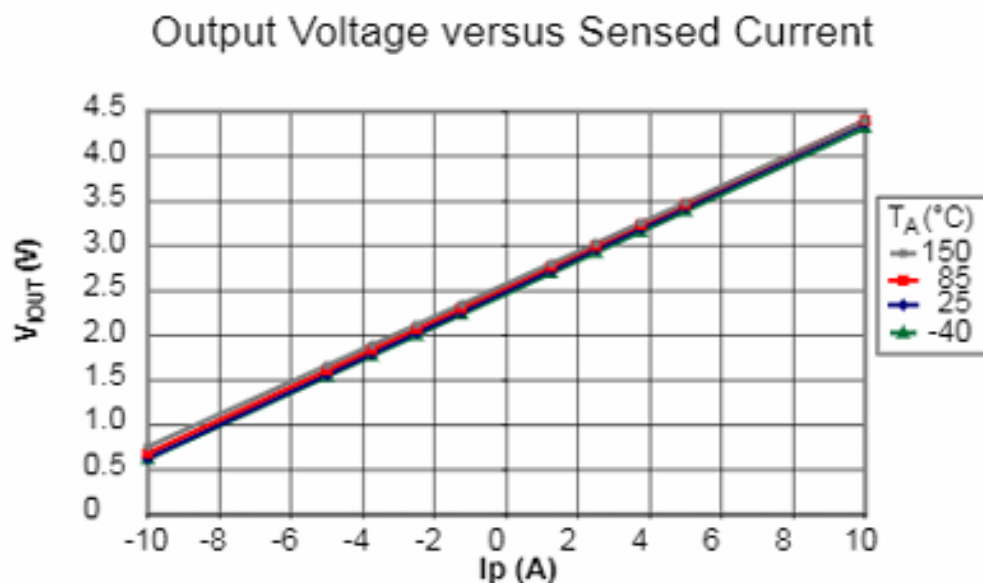


Figura 18. Salida de voltaje vs corriente medida por el sensor.

3.2.2 Conexiones Sensor ACS712

A continuación, se presentan las conexiones del sensor ACS712 con el micro-controlador ESP32 y los motores paso a paso:

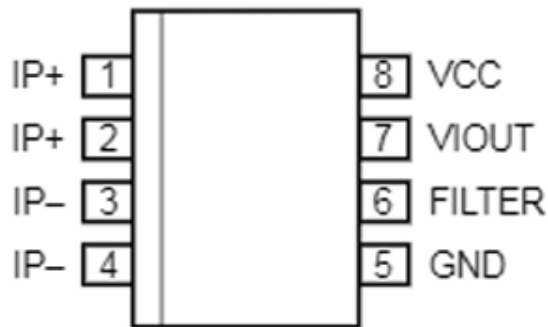


Figura 19. Distribución conexiones sensor ACS712.

Pin del ACS712	Función	Conexión en el ESP32
VCC	Alimentación	5V
GND	Tierra	GND
OUT	Salida Analógica	GPIO 34 (entrada ADC)
IP+	Entrada de Corriente+	Conexión a 12V
IP-	Entrada de Corriente-	Alimentación L298N

Tabla 4. Conexiones sensor ACS712.

3.3 MOTORES PASO A PASO BIFÁSICOS

Los motores paso a paso son dispositivos electromecánicos que convierten impulsos eléctricos en movimientos mecánicos discretos. Son ampliamente utilizados en aplicaciones que requieren movimientos precisos y controlados, como la robótica, la impresión 3D y la automatización industrial. En este proyecto se han implementado motores paso a paso para el control de la longitud de los radios de la rueda inteligente.

Los motores paso a paso modelo 17HS15-1684S-PG5 proporciona un control preciso del movimiento. Estos motores se utilizan ampliamente en aplicaciones de automatización y robótica debido a su capacidad para ofrecer pasos definidos y un posicionamiento exacto.



Figura 20. Motores paso a paso modelo 17HS15-1684S-PG5.

3.3.1 Características del Motor Paso a Paso 17HS15-1684S-PG5

Características	Descripción
Tipo de Motor	Motor paso a paso NEMA 17
Ángulo de Paso	1.8° por paso
Corriente Nominal	1.68A por fase
Voltaje Nominal	2.8V
Resistencia por Fase	1.65Ω
Par de Torsión (Torque)	44 Ncm
Reducción del Engranaje	Relación de 5:1
Dimensiones	42 x 42 x 48 mm
Peso	350g

Tabla 5. Características principales motores paso a paso.

3.3.2 Conexiones de los Motores Paso a Paso

Cada motor paso a paso está conectado al ESP32 a través de un puente H L298, lo que permite un control preciso de cada fase del motor. Las conexiones específicas para los motores son las siguientes:

Motor	Pin del Motor	GPIO del ESP32
Motor 1	IN1	GPIO 26
	IN2	GPIO 25
	IN3	GPIO 33
	IN4	GPIO 32
Motor 2	IN1	GPIO 14
	IN2	GPIO 27
	IN3	GPIO 12
	IN4	GPIO 13
Motor 3	IN1	GPIO 21
	IN2	GPIO 22
	IN3	GPIO 19
	IN4	GPIO 23

Tabla 6. Conexiones motores paso a paso.

3.4 PUENTE H L298N

El L298 es un puente H dual integrado que se utiliza para controlar cargas inductivas como motores de corriente continua y motores paso a paso como es el caso de este proyecto. Este componente es esencial en el diseño de la electrónica de control, ya que permite manejar altos voltajes y corrientes necesarios para operar los motores de manera eficiente.

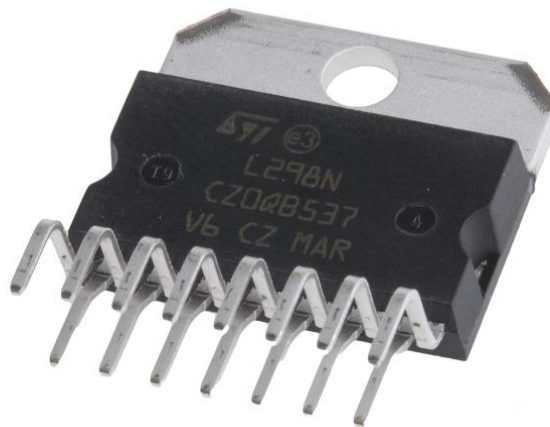


Figura 21. Puente H L298N.

3.4.1 Características del L298

Característica	Descripción
Voltaje de Operación	Hasta 46V
Corriente de Salida	Hasta 4A (total para ambos canales)
Voltaje de Entrada Lógica	4.5V a 7V
Corriente de Salida Máxima	2A por canal (repetitivo)
Protección Térmica	Sí, protección contra sobrecalentamiento
Voltaje de Saturación Bajo	Sí, baja saturación de voltaje
Paquetes Disponibles	Multiwatt15 y PowerSO20

Tabla 7. Características L298.

3.4.2 Ventajas del L298

1. Alta Capacidad de Corriente:

- El L298 puede manejar corrientes de hasta 2A por canal, lo que lo hace adecuado para controlar motores que requieren alta corriente.

2. Dual Full-Bridge Driver:

- Incorpora dos puentes H completos, permitiendo controlar dos motores de corriente continua o un motor paso a paso bipolar.

3. Protección Integrada:

- Incluye protección contra sobrecalentamiento, lo que ayuda a prevenir daños en el circuito y en los motores.

4. Compatibilidad TTL:

- Los pines de entrada y habilitación son compatibles con niveles lógicos TTL, facilitando la integración con microcontroladores como el ESP32.

3.4.3 Esquema de Conexión

En el diseño de la rueda inteligente, el L298 se utiliza para controlar los motores paso a paso que proporcionan el movimiento. El L298 permite controlar la dirección y la velocidad de los motores, facilitando un control preciso y eficiente del sistema.

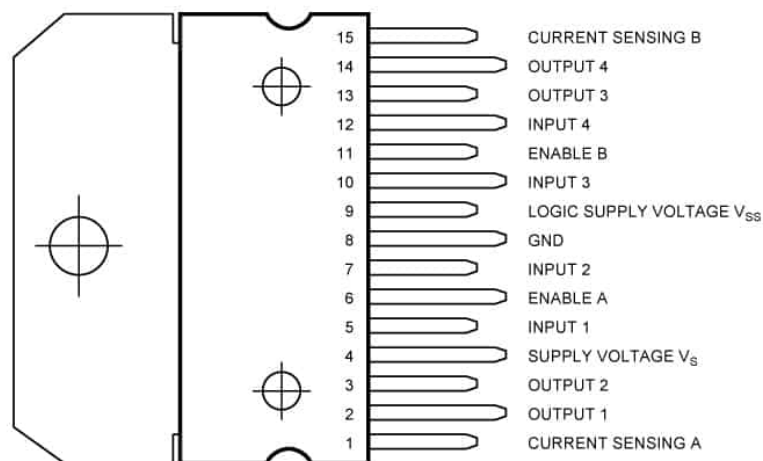


Figura 22. Esquema de conexión L298N.

Pin del L298	Descripción	Conexión
VS (Pin 4)	Alimentación de motores	Fuente de alimentación (12V)
VSS (Pin 9)	Alimentación lógica	5V
Input 1 (Pin 5)	Entrada de control puente A	GPIO 26 (Motor 1)
Input 2 (Pin 7)	Entrada de control puente A	GPIO 25 (Motor 1)
Input 3 (Pin 10)	Entrada de control puente B	GPIO 33 (Motor 1)
Input 4 (Pin 12)	Entrada de control puente B	GPIO 32 (Motor 1)
Enable A (Pin 6)	Habilitación puente A	5V
Enable B (Pin 11)	Habilitación puente B	5V
Output 1 (Pin 2)	Salida puente A	Motor IN1
Output 2 (Pin 3)	Salida puente A	Motor IN2
Output 3 (Pin 13)	Salida puente B	Motor IN3
Output 4 (Pin 14)	Salida puente B	Motor IN4
Sense A (Pin 1)	Sensor corriente puente A	GND
Sense B (Pin 15)	Sensor corriente puente B	GND

Tabla 8. Conexiones L298.

3.5 REGULADOR DE TENSIÓN

El regulador de tensión L7805CV es el encargado asegurar una alimentación estable y adecuada a los diferentes componentes del sistema, como el ESP32 y los sensores. Este regulador se utiliza para convertir una entrada de voltaje variable, en este caso 12V, a una salida constante de 5V, garantizando así que todos los componentes electrónicos reciban una tensión adecuada para su funcionamiento.

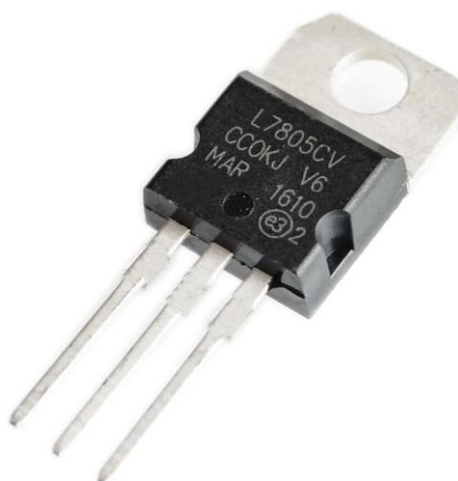


Figura 23. Regulador de tensión L7805CV.

3.5.1 Características del L7805CV

Característica	Descripción
Voltaje de Salida	5V
Rango de Voltaje de Entrada	7V a 35V
Corriente de Salida	Hasta 1.5A
Protección Térmica	Sí, incluye protección contra sobrecalentamiento
Protección de Sobrecorriente	Sí, incluye limitación de corriente
Factor de Rechazo de Ripple	62 dB
Paquete	TO-220

Tabla 9. Características L7805CV.

3.5.2 Ventajas del L7805CV

1. **Estabilidad de Voltaje:**

- El L7805CV proporciona una salida de 5V precisa y estable, esencial para el correcto funcionamiento de microcontroladores y otros componentes sensibles a las variaciones de voltaje.

2. **Alta Corriente de Salida:**

- Capaz de suministrar hasta 1.5A, el L7805CV puede alimentar múltiples componentes simultáneamente sin comprometer el rendimiento.

3. **Protección Integrada:**

- Incluye protección contra sobrecalentamiento y limitación de corriente, lo que protege tanto al regulador como a los componentes conectados contra daños debido a condiciones de sobrecarga.

4. **Simplicidad en el Diseño:**

- El L7805CV es fácil de integrar en circuitos electrónicos debido a su configuración de tres pines (entrada, tierra, salida), y requiere pocos componentes externos para su operación.

3.5.3 Esquema de Conexión

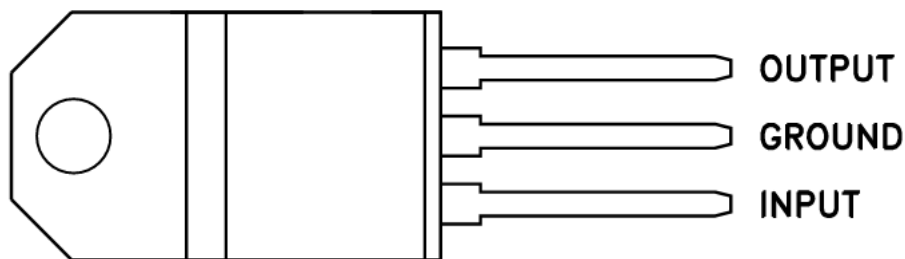


Figura 24. Conexiones L7805CV.

El L7805CV se conecta típicamente de la siguiente manera:

1. **Pin de Entrada (VIN):**

- Conectado a la fuente de alimentación principal, que puede ser una batería o un adaptador de corriente que suministre entre 7V y 35V.

2. **Pin de Tierra (GND):**

- Conectado a la tierra común del circuito.

3. Pin de Salida (VOUT):

- Proporciona una salida regulada de 5V que se distribuye a los componentes del sistema.
- **Condensador de Entrada:** Un condensador de 220 μ F conectado entre VIN y GND.

3.6 RASPBERRY PI 4

La Raspberry Pi es una microcomputadora de bajo costo y alto rendimiento, ampliamente utilizada en aplicaciones de automatización, IoT (Internet de las Cosas) y proyectos educativos. En el contexto de este proyecto, la Raspberry Pi se emplea como broker para la comunicación Wi-Fi de los distintos microcontroladores ESP32 que forman el circuito. Para lograr esto, se ha instalado el servidor MQTT Mosquitto y se utiliza Node-RED para la gestión de la información otorgada y dirigida entre los ESP32.



Figura 25. Raspberry Pi 4.

3.6.1 Características de la Raspberry Pi

Características	Descripción
Procesador	Quad-core ARM Cortex-A72 a 1.5GHz
Memoria RAM	2GB, 4GB, o 8GB LPDDR4
Almacenamiento	MicroSD, USB 3.0
Conectividad	Wi-Fi 802.11ac, Bluetooth 5.0, Ethernet Gigabit
Puertos	4 x USB (2 x USB 3.0, 2 x USB 2.0), HDMI, GPIO
Sistema Operativo	Raspbian (basado en Debian)

Tabla 10. Características Raspberry Pi 4.

3.6.2 Función en el Proyecto

En este proyecto, la Raspberry Pi se configura como el centro de comunicación (broker) para los microcontroladores ESP32. A continuación se detallan las configuraciones y funciones específicas que desempeña la Raspberry Pi en este sistema.

1. Servidor MQTT Mosquitto:

- **Instalación y Configuración:** Mosquitto es un servidor MQTT ligero que facilita la comunicación entre dispositivos IoT. Se ha instalado en la Raspberry Pi para gestionar los mensajes publicados y suscritos por los ESP32.
- **Función:** Actúa como intermediario, recibiendo mensajes de los ESP32 y distribuyéndolos a los suscriptores pertinentes, asegurando una comunicación eficiente y fiable.

2. Node-RED para la Gestión de Información:

- **Instalación y Configuración:** Node-RED es una herramienta de programación visual que permite conectar dispositivos, APIs y servicios en línea mediante un editor basado en flujo.
- **Función:** Node-RED se utiliza para crear flujos de trabajo que procesan y redirigen la información entre los ESP32. Por ejemplo, puede tomar datos de sensores enviados por un ESP32, procesarlos y enviarlos a otro ESP32 o a una base de datos para su almacenamiento y análisis.

4 DISEÑO HARDWARE

En este apartado se tratará el proceso de diseño hardware para el control de la rueda inteligente. Para el diseño hardware se ha utilizado Altium Designer, un software líder en el sector de diseño de PCBs. (Placas de Circuito Impreso).

4.1 INTRODUCCIÓN A ALTIUM DESIGNER

Altium Designer es una herramienta de diseño electrónico profesional que integra todas las etapas del diseño de PCBs en un solo entorno. Sus capacidades incluyen la captura de esquemáticos, diseño de PCB, simulación de circuitos, y generación de archivos de fabricación.

4.2 DISEÑO DEL ESQUEMÁTICO

La primera etapa en el diseño del hardware es la creación del esquemático, que representa la conexión entre los componentes del circuito. Para la realización del esquemático se seguirán las conexiones descritas en el capítulo anterior añadiendo algunos elementos extra.

4.2.1 Componentes y librerías

Previo a la creación del esquemático, es fundamental incluir en la librería los componentes necesarios para la confección del circuito. Estos archivos abarcan el esquema del componente, sus características, footprint, modelo 3D, fabricante, entre otros detalles cruciales.

Dada la amplia variedad de componentes y fabricantes disponibles, es esencial considerar varios factores al seleccionar componentes específicos de un fabricante.

Selección de Proveedores

El primer factor que considerar es el proveedor de componentes electrónicos con el que se va a trabajar. En este proyecto, se ha elegido Mouser Electronics como el proveedor principal. Es importante que todos los componentes incluidos en el diseño

estén disponibles en Mouser. Esto evita la necesidad de realizar pedidos a múltiples distribuidores, lo cual aumentaría los costos debido a los gastos de envío separados.

Ciclo de Vida de los Componentes

El segundo aspecto crucial es el ciclo de vida del componente. Los componentes electrónicos tienen un ciclo de vida que determina el tiempo que permanecen en el mercado y continúan siendo fabricados. Al diseñar un circuito, es vital asegurarse de que el componente seleccionado no esté al final de su ciclo de vida, véase figura 22. Utilizar componentes que se encuentren al principio o en la mitad de su ciclo de vida garantiza disponibilidad futura, evitando la necesidad de buscar reemplazos.




Las imágenes son sólo para referencia
Ver especificaciones del producto

Compartir

Mouser REF:	942-IRLZ44NSTRPBF
Fabr. N.º:	IRLZ44NSTRPBF
Fabr.:	Infineon Technologies
Ref. Cliente:	Ref. Cliente
Descripción:	MOSFET MOSFT 55V 47A 22mOhm 32nC Log Lvl
Ciclo de vida:	LC NRND: No se recomienda para nuevos diseños.
Hoja de datos:	IRLZ44NSTRPBF Hoja de datos (PDF)
Modelo ECAD:	 Símbolo, tamaño y modelo 3D de PCB
Descargue el Cargador de bibliotecas gratuito para convertir este archivo para su herramienta ECAD. Obtenga más información del modelo ECAD.	

☐ Comparar producto

[Añadir a proyecto](#) | [Añadir notas](#)

Figura 26. Ejemplo de componente al final de su ciclo de vida.

Verificación en Altium Designer

Una vez seleccionado un componente que cumpla con los requisitos del proveedor y el ciclo de vida, es necesario verificar su disponibilidad en Altium Designer. Se debe comprobar si el componente cuenta con el footprint, características y modelo 3D necesarios. Si el componente no sigue estándares de encapsulado comunes, como algunos circuitos integrados o resistencias, podría generar problemas. En tal caso, sería necesario crear manualmente el footprint, lo cual incrementa el riesgo de errores en el

diseño. A continuación, e las figuras 27 y 28 se muestra un ejemplo de un footprint y un modelo 3D, en este caso del L208N

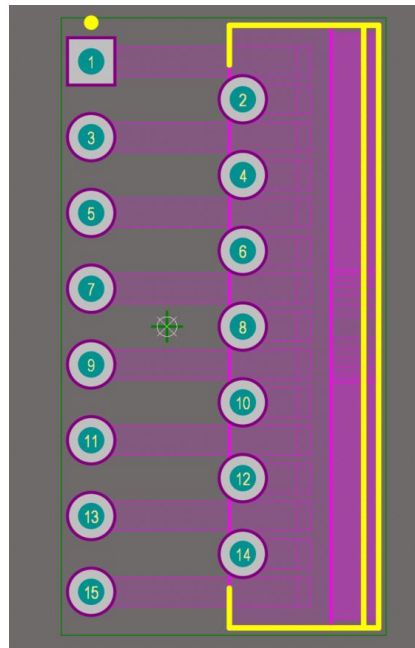


Figura 27. Footprint L298N.

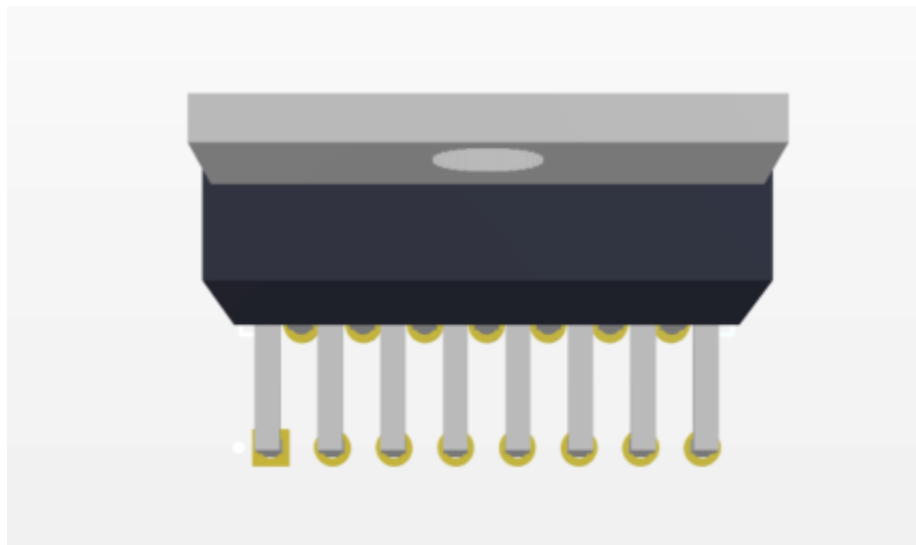


Figura 28. Modelo 3D L298N.

4.2.2 Creación del Esquemático:

Una vez tenemos todos los componentes en la librería procedemos a realizar las conexiones de los componentes descritos en el capítulo 3. Para una mayor organización y entendimiento del esquema del circuito de control de la rueda inteligente se ha realizado una división del esquema en cuatro hojas distintas, véase en anexo 1, separando los componentes según la función que desempeñan.

Alimentación y Controlador de Tensión

En esta hoja encontramos los componentes encargados de la alimentación del circuito, por una parte, tenemos la entrada de alimentación de 12V, en la cual se ha utilizado bloques terminales de dos pines para la entrada de 12V y GND como se observa en la figura 29.



Figura 29. Bloques terminales 2 posiciones.

En esta hoja también se ha incluido el controlador de tensión L7805CV encargado de convertir la tensión de entrada de 12V a 5V para alimentar el microcontrolador ESP32 y los sensores de corriente.

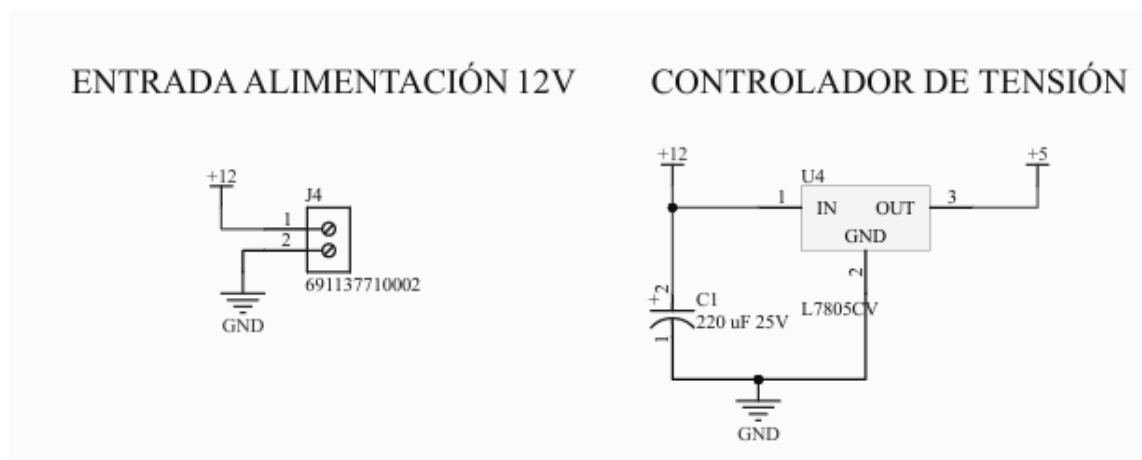


Figura 30. Esquema alimentación y controlador de tensión.

Controladores de Motores Paso a Paso

En la segunda hoja se encuentran los L298N y los diodos rectificadores. Para la conexión de la PCB con los motores paso a paso se han utilizado bloques terminales genéricos ya que el conector específico de los motores paso a paso estaba cortado y se había realizado un empalme a cables Dupont.

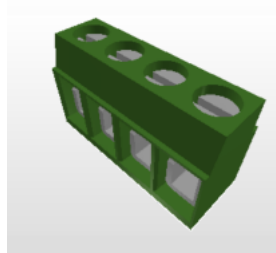


Figura 31. Bloques terminales de 4 posiciones.

En la figura 32 se observa el esquema de conexión de los L298H

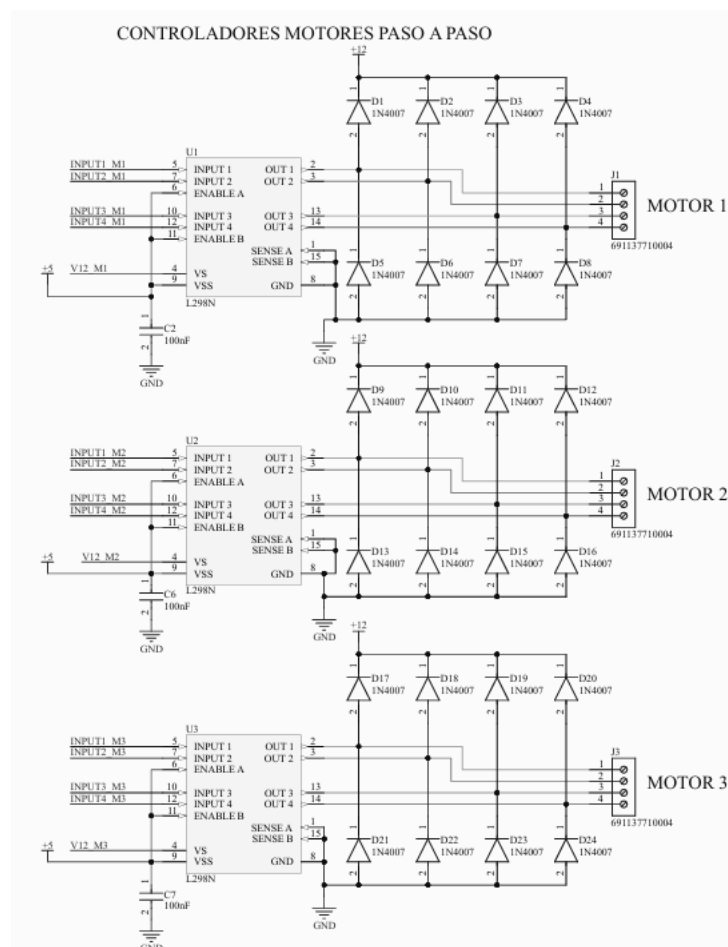


Figura 32. Controladores motores paso a paso.

Sensores de corriente

En la tercera hoja tenemos los sensores de corriente que miden el consumo de los motores para de este modo poder realizar un controlador con un PID implementado mediante software.

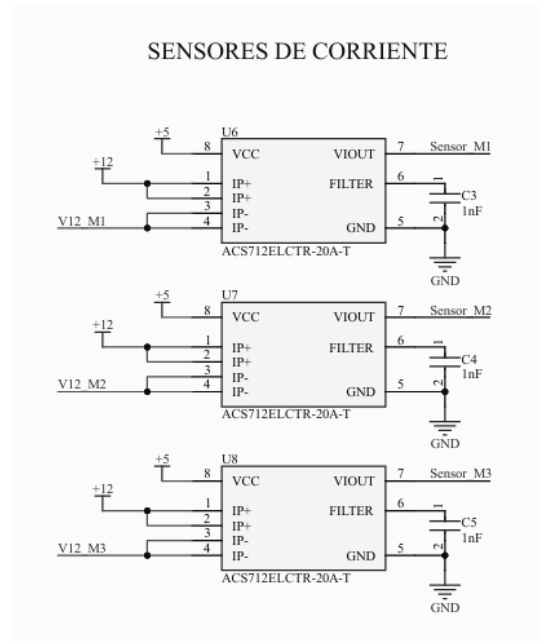


Figura 33. Sensores de corriente.

Microcontrolador ESP32

Finalmente, tenemos el esquema de conexión del microcontrolador ESP32 con las distintas partes del circuito.

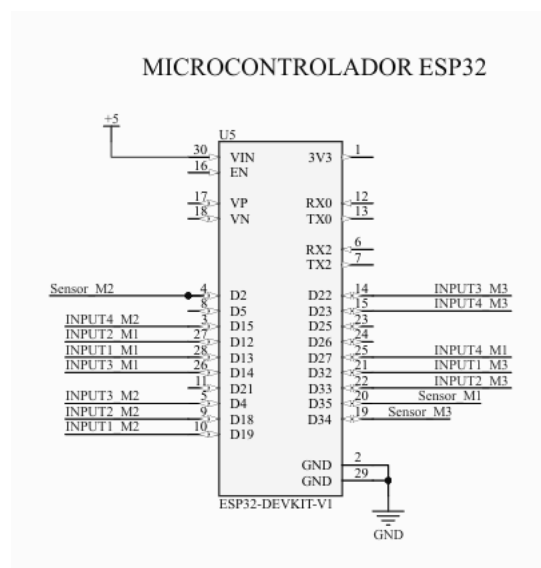


Figura 34. Microcontrolador ESP32.

4.3 DISEÑO DE LA PCB

Una vez que el esquemático está completo y verificado, el siguiente paso es diseñar la PCB, donde los componentes se colocarán físicamente y se enrutarán las conexiones eléctricas.

4.3.1 Reglas de diseño

Para asegurar que el diseño de la PCB sea manufacturable y cumpla con las especificaciones del fabricante, en el caso de este proyecto es Eurocircuits, es esencial seguir una serie de reglas de diseño. Estas reglas ayudan a minimizar errores y optimizar el proceso de fabricación. A continuación, se detallan las reglas de diseño específicas para una PCB de clase 3 a dos capas, según las directrices de Eurocircuits.

Clasificación del Patrón

La clasificación del patrón define los tamaños mínimos permitidos para las pistas de cobre (conductor) y las separaciones (aislamiento) tanto para las capas exteriores como interiores.

- **Capas Exteriores:**
 - OTT (Track to Track): 0.25 mm (250 μ m)
 - OTP (Track to Pad): 0.25 mm (250 μ m)
 - OPP (Pad to Pad): 0.25 mm (250 μ m)
 - OTW (Track Width): 0.25 mm (250 μ m)
- **Capas Interiores:**
 - ITT (Track to Track): 0.25 mm (250 μ m)
 - ITP (Track to Pad): 0.25 mm (250 μ m)
 - IPP (Pad to Pad): 0.25 mm (250 μ m)
 - ITW (Track Width): 0.25 mm (250 μ m)

Anillo Anular Mínimo

El anillo anular es la distancia mínima entre el borde de un orificio y el borde de la almohadilla de cobre que lo rodea.

- **Anillo Anular Exterior (OAR):** 0.20 mm (200 μ m)
- **Anillo Anular Interior (IAR):** 0.20 mm (200 μ m)

Para los orificios no chapados (NPTH), se recomienda un anillo anular mínimo de 0.20 mm para asegurar la integridad mecánica durante la soldadura y el uso normal.

Tamaños de taladros

La clase de taladro se basa en el tamaño más pequeño de la herramienta de producción utilizada para fabricar la placa.

- **Tamaño mínimo de herramienta (TOOLSIZ):** 0.30 mm (300 μ m)

Separación Mínima

La separación mínima entre los bordes de la placa y cualquier patrón de cobre es crucial para evitar cortocircuitos y garantizar una fabricación segura.

- **Placas fresadas:**
 - 0.25 mm (10 mil) en capas exteriores
 - 0.40 mm (16 mil) en capas interiores
- **Placas con ranurado (V-cut):**
 - 0.45 mm (18 mil) en capas exteriores e interiores

Textos y Serigrafías

Todo el texto colocado en una capa de cobre debe cumplir con las reglas de diseño para la clase dada, en el caso de este proyecto la clase 3. El texto en la capa superior de la placa debe ser legible y el texto en la capa inferior debe ser no legible o espejado para seguir la práctica universal de visualización de PCBs.

Planos de Cobre y Áreas Grandes

- **Relleno de Áreas de Cobre:**
 - Utiliza contornos o polígonos para construir áreas o planos de cobre en lugar de pequeños trazos ("painting").
 - Evita áreas pequeñas de cobre no conectadas o redes estrechas de cobre que puedan causar problemas durante la producción.

Todas estas reglas de diseño han sido implementadas en Altium Designer para asegurar que la PCB diseñada sea compatible con la clase básica de Eurocircuits, evitando así posibles problemas durante el proceso de fabricación o un aumento en el coste de la producción. Es fundamental adherirse estrictamente a estas reglas porque, en caso de incumplir alguna de las normas establecidas para la clase básica, la producción deberá ajustarse a las especificaciones de una clase superior, lo cual implicará costos adicionales significativos. Además, el incumplimiento de estas normas podría resultar en defectos o fallos en la fabricación de la PCB, lo que afectaría la fiabilidad y funcionalidad del producto final.

4.3.2 Dimensiones de la PCB

Las dimensiones de la PCB en este proyecto están restringidas debido a su ubicación en las ruedas, lo que limita las dimensiones máximas a 15 cm x 6.7 cm. La PCB diseñada mide 8.331 cm x 6.68 cm, ajustándose dentro de estas limitaciones para asegurar un encaje adecuado y funcionalidad óptima en el espacio disponible.

Es importante considerar que Eurocircuits, el fabricante seleccionado para este proyecto también impone restricciones en el tamaño de las PCBs que pueden producir. La especificación máxima de tamaño de PCB que Eurocircuits puede manejar es significativamente mayor que las dimensiones requeridas para este proyecto, lo cual asegura que el diseño actual, con sus dimensiones de 8.331 cm x 6.68 cm, se encuentra bien dentro de las capacidades de fabricación del proveedor.

En la figura 35 observamos la PCB con las contas que muestran sus dimensiones en milímetros

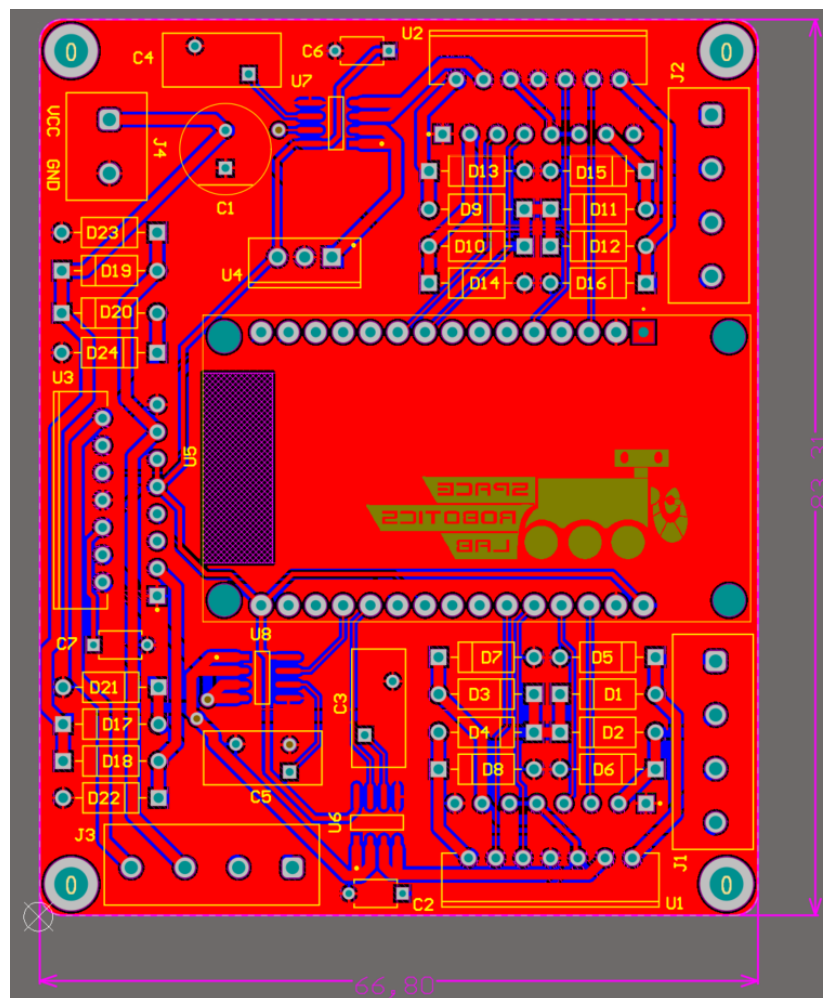


Figura 35. Dimensiones PCB.

4.3.3 Colocación de componentes en la PCB:

La colocación de los componentes en la PCB ha sido un proceso iterativo, reflejado en la realización de hasta cinco versiones anteriores antes de alcanzar el diseño final. Cada versión se desarrolló con el objetivo de optimizar la disposición de los componentes para asegurar la funcionalidad, la eficiencia de espacio y la facilidad de ensamblaje. Los componentes principales, como el microcontrolador ESP32, los sensores de corriente ACS712, el regulador de tensión L7805CV y el puente H L298, se han posicionado estratégicamente para minimizar la longitud de las trazas y reducir la interferencia electromagnética.

En la primera versión que vemos en la figura 36 se obtuvo un diseño que no respetaba las dimensiones máximas de la PCB y tampoco contemplaba el dimensionamiento del grosor de las vías.

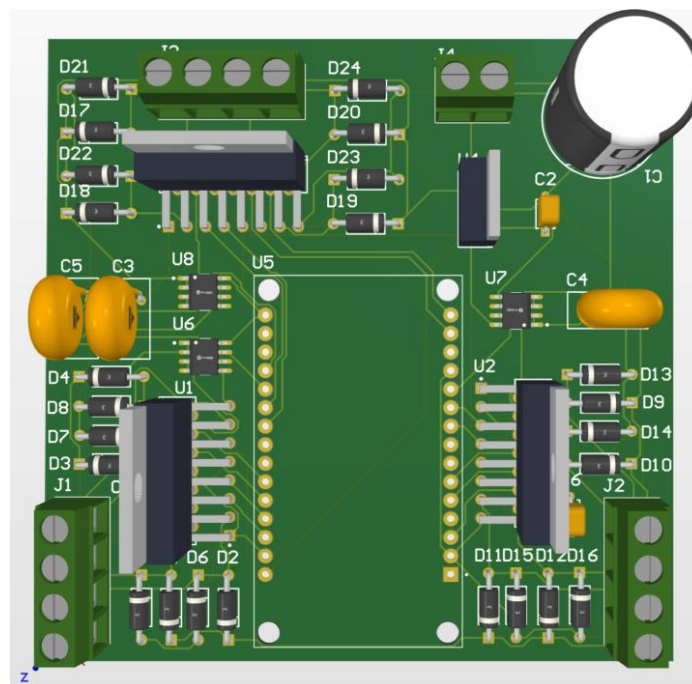


Figura 36. Primera versión PCB IMPASS.

En las versiones posteriores si se tuvieron en cuenta las dimensiones máximas exigidas y se realizó el dimensionamiento de las pistas como ya veremos en apartados posteriores.

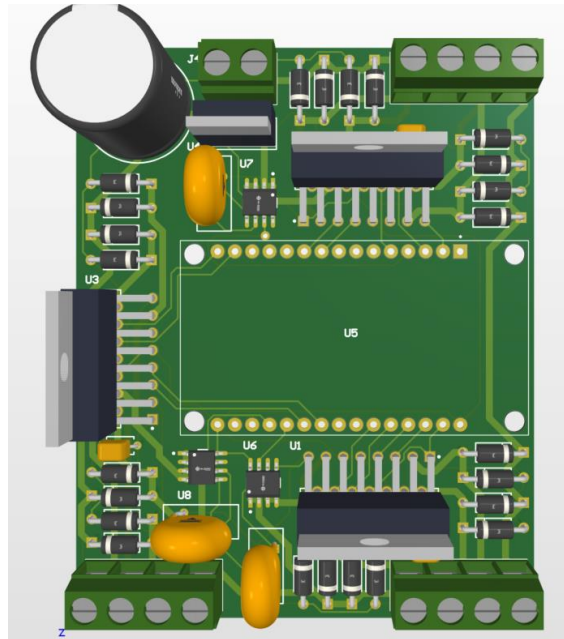


Figura 37. Tercera versión de PCB IMPASS.

Por último, tenemos la versión final de la colocación de los componentes, en la que se ha colocado los puentes H L298N en los extremos, de esta forma existe espacio suficiente para colocar disipadores ya que es un componente que tiene a calentarse. Además, se han añadido talados en las esquinas para fijar la PCB y se han redondeado los contornos.

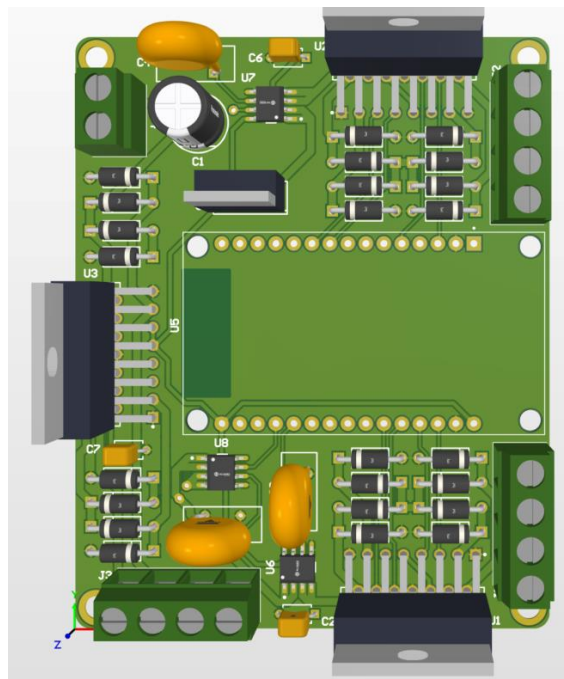


Figura 38. Versión final de PCB IMPASS.

4.3.4 Dimensionamiento de pistas

El dimensionamiento de las pistas en el diseño de la PCB ha seguido las directrices establecidas por el estándar IPC-2221B, que proporciona recomendaciones para el ancho de las pistas en función de la corriente que deben transportar. Para garantizar la fiabilidad y seguridad del circuito, se han asignado diferentes anchos de pista para las distintas tensiones de operación del sistema, basándonos en las necesidades de corriente y las capacidades térmicas de la PCB.

Para las pistas que transportan 12V, se ha utilizado un ancho de 1 mm. Este dimensionamiento permite manejar corrientes elevadas sin riesgo de sobrecalentamiento, asegurando una adecuada disipación térmica y minimizando la caída de tensión. La elección de 1 mm de ancho sigue las recomendaciones del IPC-2221B para pistas que transportan corrientes significativas en la capa exterior de la PCB, lo que es crucial para mantener la integridad del sistema de alimentación principal. Estas pistas deben soportar la corriente suministrada a los motores paso a paso para su correcto funcionamiento.

Para las pistas de 5V, se ha asignado un ancho de 0.5 mm. Este valor es adecuado para las corrientes moderadas asociadas con la alimentación del microcontrolador ESP32 y los tres sensores. La reducción del ancho respecto a las pistas de 12V es justificada por la menor corriente transportada, manteniendo aun así una capacidad térmica suficiente para evitar el sobrecalentamiento.

Finalmente, las pistas que transportan 3.3V se han dimensionado a 0.254 mm (10 mils). Este ancho es adecuado para las corrientes bajas típicas de señales y comunicaciones en circuitos de bajo consumo como son las señales de salida del microcontrolador. La elección de este ancho más estrecho permite un diseño más compacto y flexible, sin comprometer la integridad del suministro de energía para componentes sensibles.

4.3.5 Enrutamiento de pistas

El enrutado de pistas es una etapa crítica en el diseño de PCBs, especialmente a la hora de minimizar la interferencia electromagnética (EMC) y garantizar el rendimiento eléctrico óptimo del circuito. A continuación, se detallan las técnicas utilizadas para el enrutado de pistas en este proyecto, con un enfoque particular en la protección EMC.

Enrutado vertical y horizontal

Para reducir la interferencia electromagnética se ha enrutado las pistas en diferentes direcciones en las capas superior (top) e inferior (bottom) de la PCB. Las pistas se enrutan verticalmente en la capa superior y horizontalmente en la capa inferior. Este patrón cruzado ayuda a minimizar el acoplamiento inductivo y capacitivo entre las pistas de diferentes capas y ayuda para la organización de las pistas.

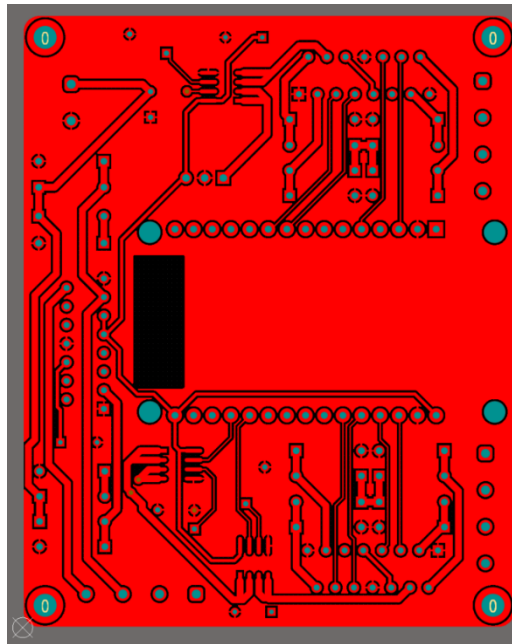


Figura 39. Enrutamiento top.

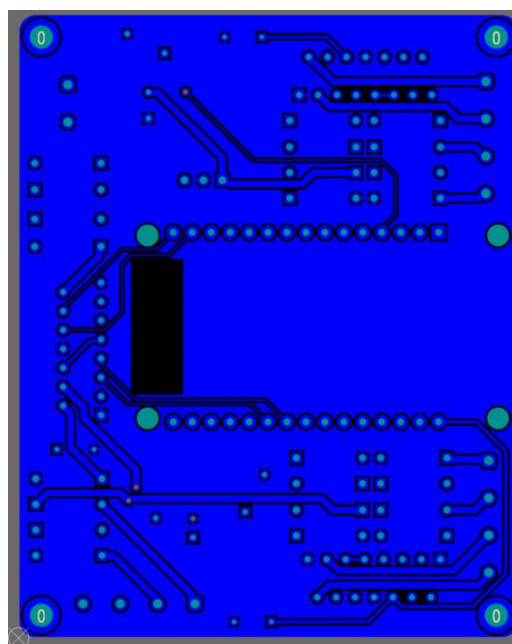


Figura 40. Enrutamiento bottom.

Planos de Tierra Continuos

Se han colocado dos planos de tierra, uno en la capa superior (top) y otro en la capa inferior (bottom). Esta configuración proporciona varias ventajas significativas:

- Mejora del Retorno de Corriente: Tener planos de tierra en ambas capas asegura que las corrientes de retorno tengan caminos de baja impedancia en cualquier capa, lo que reduce la resistencia y mejora la disipación del calor.
- Reducción de Interferencias: Los planos de tierra dobles actúan como escudos, disminuyendo la susceptibilidad del circuito a interferencias electromagnéticas externas y reduciendo la radiación de interferencias generadas internamente.
- Integridad de la Señal: Los planos de tierra continuos en ambas capas ayudan a mantener la integridad de la señal al proporcionar una referencia estable para las señales que atraviesan la PCB, lo que es especialmente crucial para señales de alta velocidad.

Desacoplo y Filtrado:

Otra de las técnicas para la protección de EMC son los condensadores de desacoplo colocándolos cerca de los pines de alimentación de los componentes críticos ya que ayuda a filtrar el ruido y estabilizar la alimentación.

4.3.6 Serigrafía

La serigrafía es una herramienta útil para la identificación de los componentes sobre todo en el ensamblado cuando se hace forma manual como es el caso. Pero también se utiliza para añadir detalles como la especificación de las señales de entrada de VCC y GND para evitar polarización inversa lo que podría suponer daños en los componentes de la PCB.

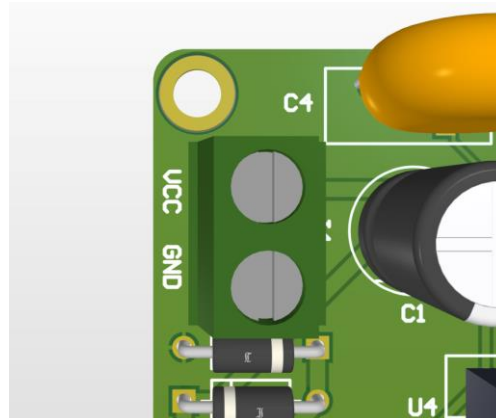


Figura 41. Serigrafía de alimentación en PCB.

También se han añadido detalles estéticos como el logo del laboratorio en el que se ha llevado a cabo el presente trabajo de fin de máster.

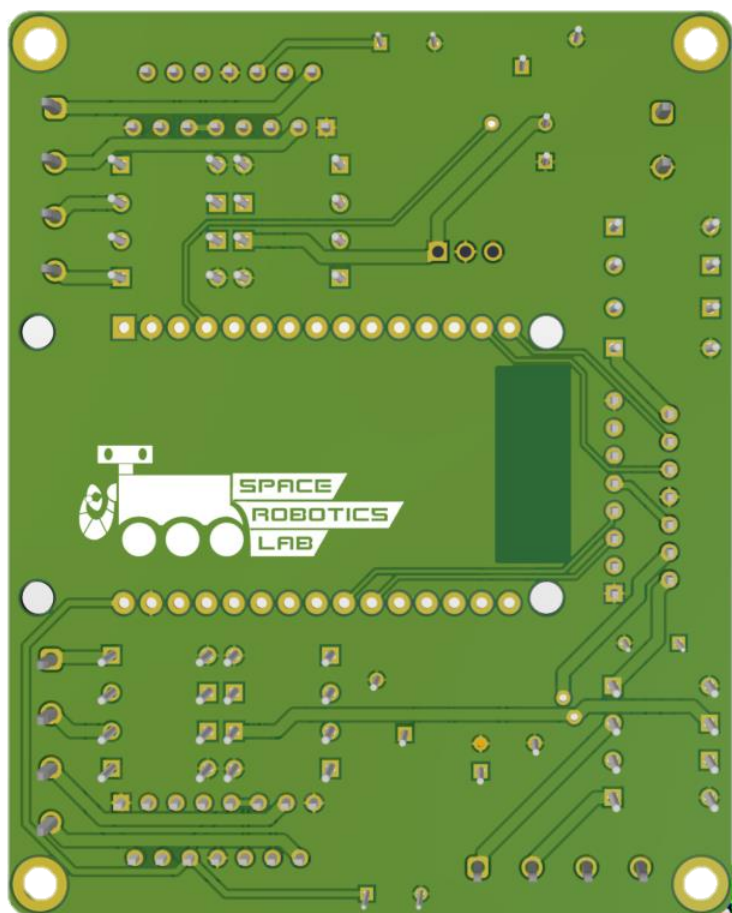


Figura 42. Serigrafía laboratorio en PCB.

4.3.7 Verificación y Simulación

Antes de enviar el diseño a fabricación, es esencial realizar verificaciones y simulaciones para asegurar que el diseño cumple con todas las especificaciones y no contiene errores.

Altium
Designer

Design Rule Verification Report

Date: 23/06/2024
Time: 20:18:10
Elapsed Time: 00:00:01
Filename: C:\Users\joseh\OneDrive\Escritorio\TFM\ALTUM\IMPASS_V1\PCB_IMPASS_Final\PcbDoc

Warnings: 0
Rule Violations: 0

Summary

Figura 43, Desing Rule Check.

4.4 GENERACIÓN DE ARCHIVOS DE FABRICACIÓN

El último paso en el diseño del hardware es generar los archivos necesarios para la fabricación de la PCB. Estos archivos incluyen los Gerbers, listas de materiales (BOM), y archivos de ensamblaje.

4.4.1 BOM

La Lista de Materiales (BOM, por sus siglas en inglés) detalla todos los componentes necesarios para ensamblar la placa, incluyendo sus designadores, descripciones, cantidades, números de parte del fabricante, y los proveedores. En este proyecto, se ha elaborado una BOM precisa y completa utilizando Altium Designer, véase tabla 11. Esta lista asegura que todos los elementos necesarios estén disponibles y correctamente especificados, minimizando errores y optimizando el proceso de ensamblaje y producción.

Tabla 11. BOM

Mouser REF	Fabr. Nº	Comment	Description	Designator	Quantity (for 1 PCB)	Total Quantity	Link
647-UCS2C221MHD1TN	UCS2C221 MHD1TN	220uF	CAP ALUM 220UF 20% 160V RADIAL	C1	1	2	Enlace compra
81-RCER71H104K0K1H3B	RCER71H10 4K0K1H03B	100nF	CAP CER 0.1UF 50V X7R RADIAL	C2, C6, C7	3	6	Enlace compra
594-F102K43Y5RP6UK5R	F102K43Y5 RP6UK5R	1nF	F102K43Y5RP6UK5R Ceramic Capacitor, Ceramic, 2000V, 10% +Tol, 10% -Tol, Y5R, 15% TC, 0.001uF, Through Hole Mount	C3, C4, C5	3	6	Enlace compra
512-UF4007	UF4007	1N4007	Rectifier diode 1000 V 1 A DO-41, ,	D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D19, D20, D21, D22, D23, D24	24	48	Enlace compra
710-691137710004	691137710 004	691137710 004	Bloque de terminal 4 pines	J1, J2, J3	3	6	Enlace compra
710-691137710002	691137710 002	691137710 002	Bloques de terminal 2 pines	J4	1	2	Enlace compra
511-L298	L298N	L298N	Dual Full-Bridge Driver, 4.8 to 46 V, -40 to 150 degC, 15-Pin FM (MW 15L), RoHS, Tube	U1, U2, U3	3	6	Enlace compra
511-L298	L298N	L7805CV	Positive Voltage Regulator, 5V, 3-Pin TO-220	U4	1	2	Enlace compra
		ESP32-DEV-KIT-V1	Dual core, Wi-Fi: 2.4 GHz up to 150 Mbits/s,BLE (Bluetooth Low Energy) and legacy Bluetooth, 32 bits, Up to 240 MHz	U5	1	2	Enlace compra
250-712ELCTR20AT	ACS712ELC TR-20A-T	ACS712ELC TR-20A-T	SENSOR CURRENT HALL 20A AC/DC	U6, U7, U8	3	6	Enlace compra

4.4.2 Gerber:

Los ficheros Gerber son un estándar de archivos utilizado en la fabricación de placas de circuitos impresos (PCBs), en este proyecto se han facilitado estos archivos a Eurocircuits. Estos archivos contienen información precisa sobre las capas individuales de la PCB, como la capa de cobre, la capa de máscara de soldadura y la capa de serigrafía. En las figuras 44 y 45 observamos el PCB top y bottom.

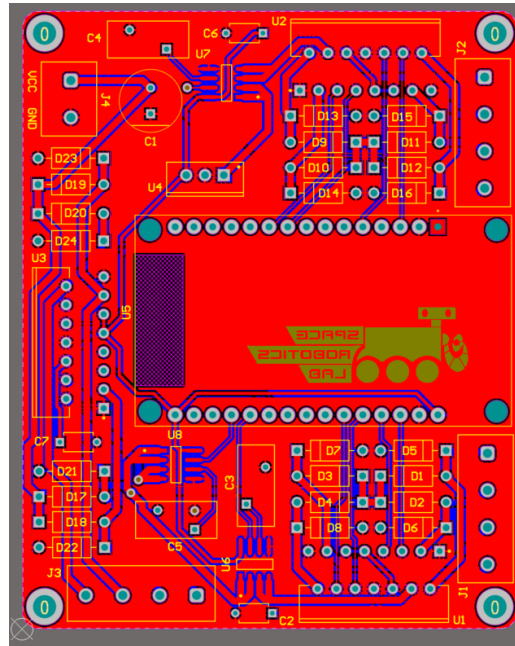


Figura 44. PCB Top.

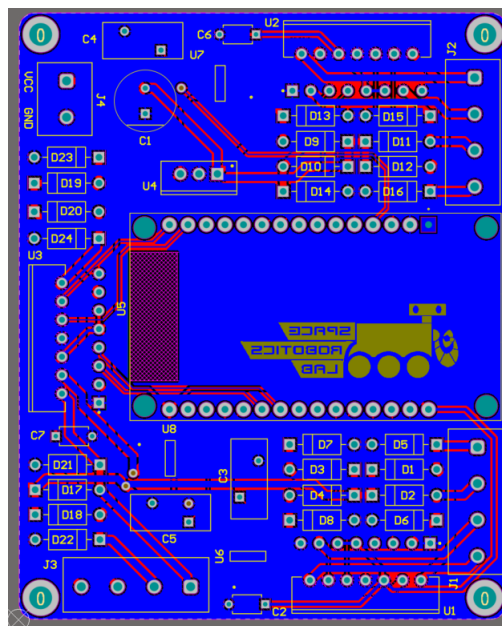


Figura 45. PCB Bottom.

4.5 MONTAJE DE COMPONENTES

El proceso de montaje de la PCB para este proyecto se ha seguido un enfoque mixto para soldar componentes Through-Hole (TH) y Surface-Mount Devices (SMD), utilizando herramientas adecuadas para cada tipo de componente.

4.5.1 Montaje de Componentes Surface-Mount (SMD)

En primer lugar, se realizó la soldadura de componentes SMD, concretamente de los sensores de corriente, lo que requiere un enfoque diferente para el montaje comparado con through-hole, utilizando pasta de soldadura y una pistola de calor.

1. Aplicación de Pasta de Soldadura:

Se ha aplicado una cantidad adecuada de pasta de soldadura en los pads correspondientes de la PCB utilizando una jeringa de pasta de soldadura asegurándose de que la pasta de soldadura esté distribuida uniformemente y cubra todos los pads de montaje.

2. Colocación de Sensores SMD:

Con la ayuda de pinzas se ha colocado cuidadosamente los sensores de corriente SMD sobre la pasta de soldadura aplicada, verificando que los sensores estén alineados correctamente con los pads de la PCB.

3. Reflujo de Soldadura:

Finalmente se utilizar una pistola de calor para calentar la pasta de soldadura hasta que se derrita y forme una unión sólida entre los pads de la PCB y los pines de los sensores.



Figura 46. Pistola de calor para soldadura SMD.

4.5.2 Montaje de Componentes Through-Hole (TH)

La mayoría de los componentes en esta PCB son Through-Hole, lo que implica que los pines de los componentes atraviesan la PCB y se sueldan en el lado opuesto. Para el proceso de montaje de estos componentes se ha seguido los siguientes pasos:

1. Inserción de Componentes TH:

En primer lugar, se inserta cada componente a través de los orificios correspondientes según la serigrafía y la BOM, asegurarse de que los componentes estén bien asentados y en la orientación correcta.

2. Soldadura de Componentes TH:

Se ha utilizado un soldador de estaño básico con una punta fina adecuada para la precisión. Aplicando calor a la unión del pin y la pista de cobre, y luego alimentando la soldadura hasta que forme una unión limpia y brillante.



Figura 47. Cautín para soldadura Through-Hole.

Inspección y Verificación

Tras completar el montaje de todos los componentes, se procede a realizar una inspección y verificación minuciosa para asegurar la calidad del montaje.

1. Inspección Visual:

Verificar visualmente todas las uniones de soldadura, asegurándose de que no haya puentes de soldadura, soldaduras frías o componentes mal alineados.

2. Pruebas de Continuidad:

Con ayuda de un multímetro para comprobar la continuidad de las pistas y asegurarse de que todas las conexiones sean correctas.

3. Pruebas Funcionales:

Por último, se realizan pruebas funcionales iniciales del circuito, moviendo los tres motores y realizando alguna medida de los sensores de corriente, para confirmar que todos los componentes están funcionando correctamente y que el diseño cumple con las especificaciones.

4.5.3 PCB montada

Se han añadido disipadores de calor a los L298N para prevenir posibles aumentos de temperatura en el componente

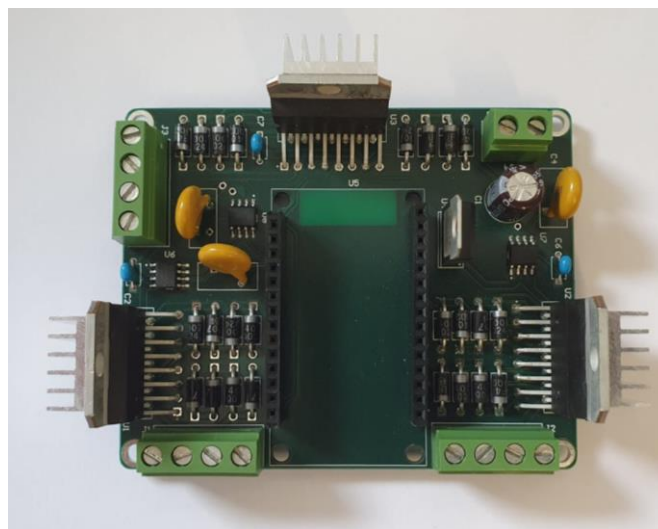


Figura 48. PCB montada 1.

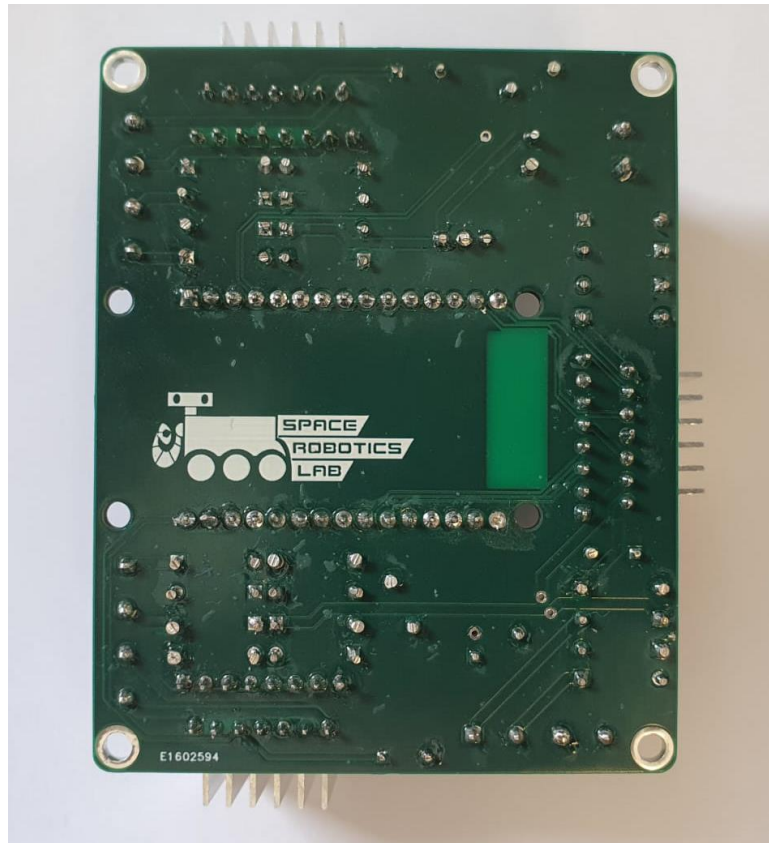


Figura 49. PCB montada 2.

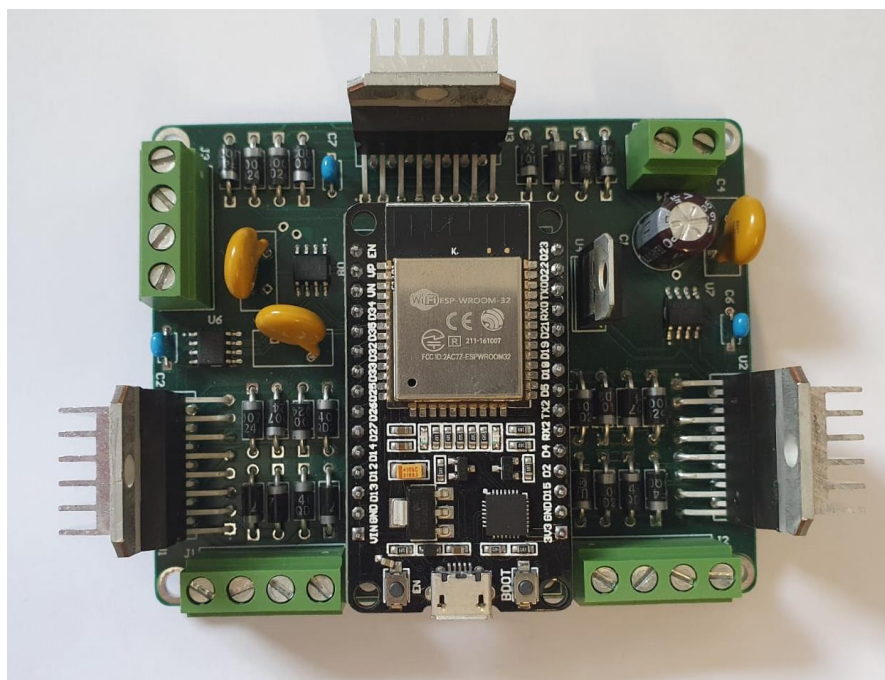


Figura 50. PCB montada con microcontrolador.

5 PROGRAMACIÓN FIRMWARE ESP32

En este apartado se describirá el funcionamiento del firmware desarrollado para el microcontrolador ESP32 que ira integrada en la PCB diseñada. La finalidad de este firmware es la de realizar el control de la longitud de los radios de la rueda inteligente, este control se realiza en base a la posición y velocidad de cada una de las ruedas del IM-PASS. Además de esto también se recogerán datos de los sensores de corriente, que proporcionarán información sobre el consumo de cada uno de los motores paso a paso. La transmisión de información entre las distintas partes del sistema se realizará con comunicación MQTT utilizando una Raspberry Pi 4 como bróker.

5.1 INTRODUCCIÓN

A continuación, se explicará a rasgos generales el algoritmo de funcionamiento del firmware apoyándose en un diagrama de flujo para un mejor entendimiento del sistema.

El microcontrolador ESP32 comienza ejecutando el Setup, donde se inician los pines que se usarán más tarde en el microcontrolador y se crean las tareas que posteriormente se paralelizarán.

1. Dentro del setup se llama a la función Setup WiFi que realiza la conexión a internet con las credenciales configuradas.
2. Posteriormente se configura la conexión al broker MQTT y se establecen los parámetros necesarios para la comunicación.
3. Se configura las funciones callbacks para manejar los mensajes recibidos a través de MQTT.

Una de las principales ventajas al escoger el ESP32 como microcontrolador es su potencia que admite la ejecución de tareas en paralelo. De este modo se aprovecha al máximo su arquitectura de doble núcleo. Gracias a esto se ejecuta de forma paralela dos bucles simultáneamente.

El primer bucle de control es el encargado de manejar la máquina de estados para el control de los motores paso a paso. Esta máquina de estados se ejecuta con la información obtenida por MQTT.

El segundo bucle de control se encarga de la comunicación MQTT (tanto envió como recepción de mensajes) y el control de la señal PWM encargada de actuar como freno cuando los motores paso a paso están en reposo.

1. Este bucle comienza su ejecución comprobando el estado de la conexión MQTT, en caso de desconexión se conecta de nuevo al bróker MQTT y se vuelve a suscribir a los topics que corresponde.
2. Si la conexión se mantiene, el sistema recibe mensajes MQTT con datos de posición y velocidad.
3. Se realiza el control PWM de los motores en reposo utilizando un controlador PID basado en las lecturas de corriente de los sensores.
4. Se leen los valores de corriente de los sensores ACS712.
5. Se envían los datos de los sensores a través de MQTT para la monitorización en tiempo real.

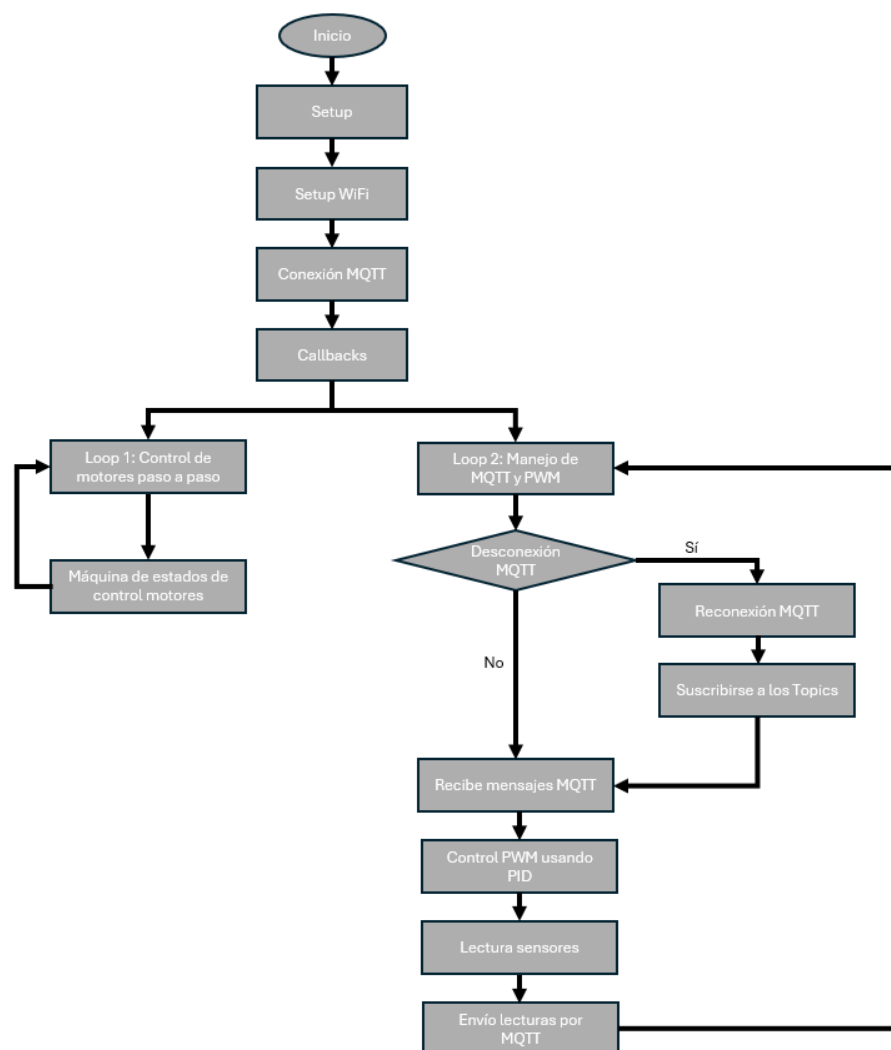


Figura 51. Diagrama de flujo del Microcontrolador.

5.2 SETUP Y CONEXIÓN WIFI

En la función “setup” es la primera función en ejecutarse cuando se inicia el microcontrolador. En esta se inicializa la comunicación serial para el envío de mensajes por puerto serie, utilizado para la depuración del código. En segundo lugar, se configura la conexión WiFi con la función “setup_wifi” que veremos más adelante y comunicación MQTT. Se establece la velocidad por defecto de los motores paso a paso y se establecen los pines de los motores y sensores como salidas o entradas. Por último, se crean las dos tareas que se quiere paralelizar, asignándole a cada una el núcleo donde se ejecutará, el tamaño de memoria de la pila y la prioridad de cada tarea.

```
void setup() {
    // Inicializa el puerto serial
    Serial.begin(115200);
    esp_task_wdt_deinit();//Eliminamos los wachgogs para que no se reinicie el micro-
    controlador antes esperas largas

    //Configuración de la comunicación MQTT
    setup_wifi();
    client.setServer(mqtt_server, 1883);
    client.setCallback(callback);

    // Establece la velocidad inicial en 60rpm
    myStepper3.setSpeed(60);
    myStepper2.setSpeed(60);
    myStepper1.setSpeed(60);

    // Declaración de pines
    pinMode(INPUT1_M1, OUTPUT);
    pinMode(INPUT2_M1, OUTPUT);
    ...
    pinMode(Sensor_M3, INPUT);

    xTaskCreatePinnedToCore(
        loop1,          // Función que implementa la tarea.
        "Task2",        // Nombre de la tarea.
        50000,          // Tamaño del stack en bytes.
        NULL,           // Parámetro pasado a la tarea.
        2,              // Prioridad de la tarea.
        NULL,           // Handle de la tarea
        0               // Número del núcleo donde se ejecutará la tarea. (El 0 es el
        que no se usa por defecto)
    );

    xTaskCreatePinnedToCore(
        loop2,          // Función que implementa la tarea.
        "Task1",        // Nombre de la tarea.
        20000,          // Tamaño del stack en bytes.
        NULL,           // Parámetro pasado a la tarea.
        1,              // Prioridad de la tarea.
        NULL,           // Handle de la tarea
        1               // Número del núcleo donde se ejecutará la tarea. (El 0 es el
        que no se usa por defecto)
    );
}
```

Código 1. setup

La utilización de paralelismo es una mejora significativa en el tiempo de ejecución en comparación al código secuencial con interrupciones que ejecutaba el Arduino mega anteriormente.

La segunda función en ejecutarse es “setup_wifi” encargada de realizar la conexión WiFi del microcontrolador. Esta función se conecta con las credenciales configuradas y muestra un punto por puerto serie hasta realizar la conexión. Es importante aclarar que se mantiene en bucle infinito hasta que no se conecte, ya que sin conexión no puede comunicarse por MQTT y por tanto no podrá realizar el control de los radios de la rueda inteligente correctamente.

```
void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi conectado");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}
```

Código 2. setup_wifi

5.3 COMUNICACIÓN MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo ligero de mensajería diseñado para la comunicación de dispositivos con recursos limitados y en entornos de red de baja latencia. En este proyecto, se utiliza MQTT para la comunicación entre el ESP32 y una Raspberry Pi, facilitando la transmisión de datos de posición y velocidad de las ruedas principales.

La implementación de MQTT en el ESP32 se lleva a cabo utilizando la biblioteca PubSubClient. A continuación, se describen las funciones clave involucradas en la configuración y gestión de la comunicación MQTT: `reconnect()`, y `callback()`.

La función “`reconnect()`” es necesaria para manejar las desconexiones y asegurar que el dispositivo siga recibiendo y enviando mensajes correctamente. Cuando el ESP32 pierde la conexión con el broker MQTT, no solo debe reconectar, sino que también debe suscribirse nuevamente a los topics necesarios.

El sistema distingue entre la rueda derecha (rueda_R) y la rueda izquierda mediante una variable booleana rueda_R. Esta variable determina a qué topics se suscribirá el ESP32 y qué mensajes procesará.

```
void reconnect() {
    // Bucle hasta que nos reconectemos
    while (!client.connected()) {
        Serial.print("Intentando conexión MQTT...");
        // Crear un ID de cliente
        String clientId;
        if (rueda_R) {
            clientId = "ESP32Client_Right-";
        } else {
            clientId = "ESP32Client_Left-";
        }
        clientId += String(WiFi.macAddress());
        // Intentar conectar
        if (client.connect(clientId.c_str())) {
            Serial.println("conectado");
            // Una vez conectado, suscribirse a los tópicos necesarios...
            if (rueda_R) {
                client.subscribe("ESP32_R/velocity");
                client.subscribe("ESP32_R/position");
            } else {
                Serial.print("suscrito");
                client.subscribe("ESP32_L/velocity");
                client.subscribe("ESP32_L/position");
            }
        } else {
            Serial.print("falló, rc=");
            Serial.print(client.state());
            Serial.println(" intentando de nuevo en 1 segundo");
            // Esperar 1 segundo antes de intentar de nuevo
            delay(1000);
        }
    }
}
```

Código 3. reconnect

La función “callback()” se encarga de procesar los mensajes recibidos de los topics suscritos. Utiliza la variable “rueda_R” para determinar a qué rueda se refiere cada mensaje.

```
void callback(char* topic, byte* payload, unsigned int length)
{
    Serial.print("Mensaje recibido en topic: ");
    Serial.print(topic);
    Serial.print(", Message: ");

    if(rueda_R)//Rueda derecha
    {
        if (strcmp(topic, "ESP32_R/position") == 0)//Posición de la rueda en rad[0-2pi]
        {
            // Construir un float a partir del payload recibido
            char receivedFloat[length + 1];
            memcpy(receivedFloat, payload, length);
            receivedFloat[length] = '\0'; // Terminar la cadena con null
            float receivedValue = atof(receivedFloat); // Convertir char array a float
        }
    }
}
```

```

position = receivedValue;//Posición de la rueda derecha

Serial.print("Position: ");
Serial.println(position, 2); // Imprimir con 2 decimales
}
else if (strcmp(topic, "ESP32_R/velocity") == 0)//Velocidad de la ruedas en rpm
{
    char receivedFloat[length + 1];
    memcpy(receivedFloat, payload, length);
    receivedFloat[length] = '\0'; // Terminar la cadena con null
    float receivedValue = atof(receivedFloat); // Convertir char array a float
    rpmobj = receivedValue;//Velocidad de la rueda derecha

    Serial.print("Velocity: ");
    Serial.println(rpmobj, 2); // Imprimir con 2 decimales
}
}
else//Rueda izquierda
{
    if (strcmp(topic, "ESP32_L/position") == 0)//Posición de la rueda en rad[0-2pi]
    {
        // Construir un float a partir del payload recibido
        char receivedFloat[length + 1];
        memcpy(receivedFloat, payload, length);
        receivedFloat[length] = '\0'; // Terminar la cadena con null
        float receivedValue = atof(receivedFloat); // Convertir char array a float
        position = receivedValue;//Posición de la rueda izquierda

        Serial.print("Position: ");
        Serial.println(position, 2); // Imprimir con 2 decimales
    }
    else if (strcmp(topic, "ESP32_L/velocity") == 0)//Velocidad de la ruedas en rpm
    {
        char receivedFloat[length + 1];
        memcpy(receivedFloat, payload, length);
        receivedFloat[length] = '\0'; // Terminar la cadena con null
        float receivedValue = atof(receivedFloat); // Convertir char array a float
        rpmobj = receivedValue;//Velocidad de la rueda izquierda

        Serial.print("Velocity: ");
        Serial.println(rpmobj, 2); // Imprimir con 2 decimales
    }
}
}
}

```

Código 4. Callbacks.

5.4 CONTROL DE MOTORES PASO A PASO

Para realizar el control de los motores paso a paso se utiliza la función `stepMotor`. Esta función permite mover un motor específico una cantidad determinada de pasos y, al finalizar el movimiento, aplicar un ciclo de trabajo PWM para mantener el motor en su posición y evitar el consumo de energía innecesario.

Parámetros de la Función

1. `nMotor`: Un entero que indica cuál de los tres motores paso a paso se debe controlar (1, 2 o 3).

2. nPasos: Un entero que representa el número de pasos que el motor debe realizar.
3. ductyCicle: Un entero que define el ciclo de trabajo PWM para mantener el motor en su posición después del movimiento.

Esta función se ha creado debido a la necesidad de reducir el consumo de corriente que supone el manejo de los motores paso a paso. Si solo utilizamos las funciones que incluyen la librería Steppers los motores a pesar de estas en reposos consume mucho más de lo necesario. Por eso cuando se termina de realizar el movimiento del motor, se establecen los pines de control de los motores a nivel bajo y se controla con un PWM el consumo del motor para que los radios de la rueda se mantengan en su posición y no cedan debido al peso del robot.

```
void stepMotor(int nMotor, int nPasos, int ductyCicle)
{
    switch (nMotor)
    {
        case 1:
            Serial.println("M1");
            //Reset de los pines
            pinMode(INPUT1_M1, OUTPUT);
            pinMode(INPUT2_M1, OUTPUT);
            pinMode(INPUT3_M1, OUTPUT);
            pinMode(INPUT4_M1, OUTPUT);
            digitalWrite(INPUT1_M1, LOW);
            digitalWrite(INPUT2_M1, LOW);
            digitalWrite(INPUT3_M1, LOW);
            digitalWrite(INPUT4_M1, LOW);
            //Movimiento del motor
            motor1=true;
            myStepper1.step(nPasos);
            motor1=false;
            //Apagamos los transistores para que no consuma
            digitalWrite(INPUT1_M1, LOW);
            digitalWrite(INPUT2_M1, LOW);
            digitalWrite(INPUT3_M1, LOW);
            digitalWrite(INPUT4_M1, LOW);
            //Controlamos con una PWM para frenar el motor
            analogWrite(INPUT1_M1, ductyCicle);
            break;

        case 2:
            Igual pero en el motor 2
            break;
        case 3:
            Igual pero en el motor 3
            break;
        default:
            Serial.println("Motor no válido");
            break;
    }
}
```

Código 5. ControlSteps.

5.5 LECTURA DE SENSORES

La función `readSensors` está diseñada para leer y calcular la corriente medida por tres diferentes sensores de corriente ACS712. La función toma como parámetro el número del sensor (`nSensor`) y devuelve la corriente medida por ese sensor. La función adapta su método de medición según el estado del motor (en movimiento o en reposo) para filtrar el ruido de manera eficiente. Cuando un motor está en movimiento, el ruido eléctrico generado puede afectar la precisión de las mediciones de corriente. Al restar el promedio de las lecturas de los otros dos sensores, se puede cancelar gran parte del ruido común, resultando en una lectura más precisa de la corriente del motor en movimiento.

```
float readSensors(int nSensor)
{
    float measureSensor=0;

    switch(nSensor)
    {
        case 1:
            if(motor1)
            {
                sensor1Value = analogRead(Sensor_M1)-((analogRead(Sensor_M2)+analogRead(Sensor_M3))/2);
                sensor1Voltage = (sensor1Value * 3.3) / 4096.0;
                sensor1Current = (sensor1Voltage)/0.1 ;
                measureSensor = sensor1Current;
            }
            else
            {
                sensor1Value = analogRead(Sensor_M1);
                sensor1Voltage = (sensor1Value * 3.3) / 4096.0;
                sensor1Current = (sensor1Voltage)/0.1 ;
                measureSensor = sensor1Current;
            }
            break;

        case 2:
            if(motor2)
            {
                sensor2Value = analogRead(Sensor_M2)-((analogRead(Sensor_M1)+analogRead(Sensor_M3))/2);
                sensor2Voltage = (sensor2Value * 3.3) / 4096.0;
                sensor2Current = (sensor2Voltage)/0.1 ;
                measureSensor = sensor2Current;
            }
            else
            {
                sensor2Value = analogRead(Sensor_M2);
                sensor2Voltage = (sensor2Value * 3.3) / 4096.0;
                sensor2Current = (sensor2Voltage)/0.1 ;
                measureSensor = sensor2Current;
            }
            break;

        case 3:
            if(motor3)
```

```
{
    sensor3Value = analogRead(Sensor_M3)-((analogRead(Sensor_M2)+analogRead(Sensor_M1))/2);
    sensor3Voltage = (sensor3Value * 3.3) / 4096.0;
    sensor3Current = (sensor3Voltage)/0.1 ;
    measureSensor = sensor3Current;
}
else
{
    sensor3Value = analogRead(Sensor_M3);
    sensor3Voltage = (sensor3Value * 3.3) / 4096.0;
    sensor3Current = (sensor3Voltage)/0.1 ;
    measureSensor = sensor3Current;
}
break;

default:
    Serial.println("Sensor no válido");
    break;
}
return measureSensor;
}
```

Código 6. ReadSensors.

5.6 CONTROL PID PARA PWM

La función “controlPWM” se encarga de mantener el consumo de corriente de los motores paso a paso en reposo a un valor de referencia específico (0.10 A en este caso), este valor ha sido obtenido experimentalmente para que los motores no cedan ante la fuerza del peso del robot. Utiliza un controlador PID (Proporcional-Integral-Derivativo) para ajustar el ciclo de trabajo (duty cycle) de la señal PWM aplicada a los motores.

```
void controlPWM(){
    static float lastError1 = 0, integral1 = 0;
    static float lastError2 = 0, integral2 = 0;
    static float lastError3 = 0, integral3 = 0;

    float current1 = readSensors(1);
    float current2 = readSensors(2);
    float current3 = readSensors(3);

    float kP = 1.0;
    float kI = 0.1;
    float kD = 0.01;

    // Control PID para el motor 1
    if(!motor1)//Comprobación estado reposo motor
    {
        float error1 = CURRENT_REF - current1;
        integral1 += error1;
        float derivative1 = error1 - lastError1;
        lastError1 = error1;
        float output1 = kP * error1 + kI * integral1 + kD * derivative1;
        dutyCycle1 = constrain(output1, 0, MAX_DUTY_CYCLE);
    }

    // Control PID para el motor 2
```

```

if(!motor2)
{
    float error2 = CURRENT_REF - current2;
    integral2 += error2;
    float derivative2 = error2 - lastError2;
    lastError2 = error2;
    float output2 = kP * error2 + kI * integral2 + kD * derivative2;
    dutyCycle2 = constrain(output2, 0, MAX_DUTY_CYCLE);
}

// Control PID para el motor 3
if(!motor3)
{
    float error3 = CURRENT_REF - current3;
    integral3 += error3;
    float derivative3 = error3 - lastError3;
    lastError3 = error3;
    float output3 = kP * error3 + kI * integral3 + kD * derivative3;
    dutyCycle3 = constrain(output3, 0, MAX_DUTY_CYCLE);
}
}

```

Código 7. ControlPWM.

Esta función solo se ejecuta en los motores que están en reposo. Si realizase el control sobre los motores que están en movimiento, con mayor consumo y que no son controlados por la señal PWM, el ciclo de trabajo podría tomar valores incorrectos y provocar consumos inferiores para intentar compensar la subida que supone el movimiento de los motores, lo que conllevaría a la posibilidad de que los motores cedan ante el peso del motor debido a esta señal mal configurada.

Los ciclos de trabajo calculados se guardan en variable globales para luego ser aplicados en la función de máquina de estados para el control del movimiento.

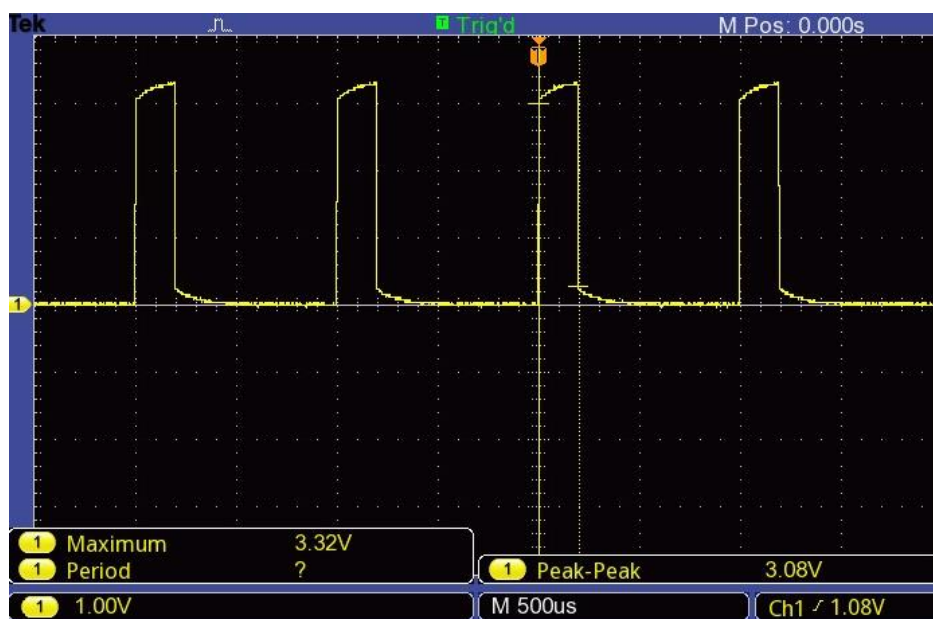


Figura 52. Señal PWM generada.

5.7 MÁQUINA DE ESTADOS PARA EL CONTROL DE MOVIMIENTO

Esta es una de las funciones principales, ya que se encarga de gestionar el movimiento de los motores paso a paso en función de la posición y velocidad de las ruedas principales. Esta implementación permite un control preciso y coordinado de los motores, asegurando que los radios de la rueda se ajusten correctamente durante la operación del robot. A continuación, se detalla la estructura y funcionamiento de la máquina de estados.

```
void controlSteps(){  
    switch(estado)  
    {  
        case 0:  
            if(inicio)  
            {  
                if(rueda_R)  
                {  
                    checkvelocities(d1,d2,d3);  
                    stepMotor(1,-pasos1,dutyCycle1);  
                    stepMotor(2,pasos2,dutyCycle2);  
                    stepMotor(3,pasos3,dutyCycle3);  
                }  
                else  
                {  
                    checkvelocities(d1,d2,d3);  
                    stepMotor(1,pasos1,dutyCycle1);  
                    stepMotor(2,-pasos2,dutyCycle2);  
                    stepMotor(3,-pasos3,dutyCycle3);  
                }  
                inicio=false;  
            }  
            if (!(position<PI/3.0))  
            {  
                estado=1;  
                inicio=true;  
                Serial.print("Voy a estado 1, posicion:");  
                Serial.println(position);  
            }  
            break;  
        case 1:  
            if(inicio){  
                if(rueda_R)  
                {  
                    checkvelocities(d2,d3,d1);  
                    stepMotor(1,pasos2,dutyCycle1);  
                    stepMotor(2,-pasos3,dutyCycle2);  
                    stepMotor(3,-pasos1,dutyCycle3);  
                }  
                else  
                {  
                    checkvelocities(d2,d3,d1);  
                    stepMotor(1,-pasos2,dutyCycle1);  
                    stepMotor(2,pasos3,dutyCycle2);  
                    stepMotor(3,pasos1,dutyCycle3);  
                }  
            }  
    }
```

```
        inicio=false;
    }

    if(!(position<(2.0*PI/3.0))){
        estado=2;
        inicio=true;

        Serial.print("Voy a estado 2, posicion:");
        Serial.println(position);
    }
    break;

case 2:
    if(inicio)
    {
        if(rueda_R)
        {
            checkvelocities(d3,d1,d2);
            stepMotor(1,-pasos3,dutyCycle1);
            stepMotor(2,pasos1,dutyCycle2);
            stepMotor(3,pasos2,dutyCycle3);
        }
        else
        {
            checkvelocities(d3,d1,d2);
            stepMotor(1,pasos3,dutyCycle1);
            stepMotor(2,-pasos1,dutyCycle2);
            stepMotor(3,-pasos2,dutyCycle3);
        }

        inicio=false;
    }

    if(!(position<PI))
    {
        estado=3;
        inicio=true;

        Serial.print("Voy a estado 3, posicion:");
        Serial.println(position);
    }
    break;

case 3:
    if(inicio)
    {
        if(rueda_R)
        {
            checkvelocities(d1,d2,d3);
            stepMotor(1,pasos1,dutyCycle1);
            stepMotor(2,-pasos2,dutyCycle2);
            stepMotor(3,-pasos3,dutyCycle3);
        }
        else
        {
            checkvelocities(d1,d2,d3);
            stepMotor(1,-pasos1,dutyCycle1);
            stepMotor(2,pasos2,dutyCycle2);
            stepMotor(3,pasos3,dutyCycle3);
        }

        inicio=false;
    }
}
```

```
if(!(position<(4.0*PI/3.0)))
{
    estado=4;
    inicio=true;

    Serial.print("Voy a estado 4, posicion:");
    Serial.println(position);
}
break;

case 4:
    if(inicio)
    {
        if(rueda_R)
        {
            checkvelocities(d2,d3,d1);
            stepMotor(1,-pasos2,dutyCycle1);
            stepMotor(2,pasos3,dutyCycle2);
            stepMotor(3,pasos1,dutyCycle3);
        }
        else
        {
            checkvelocities(d2,d3,d1);
            stepMotor(1,pasos2,dutyCycle1);
            stepMotor(2,-pasos3,dutyCycle2);
            stepMotor(3,-pasos1,dutyCycle3);
        }

        inicio=false;
    }

    if(!(position<(5.0*PI/3.0)))
    {
        estado=5;
        inicio=true;

        Serial.print("Voy a estado 5, posicion:");
        Serial.println(position);
    }
    break;

case 5:
    if(inicio)
    {
        if(rueda_R)
        {
            checkvelocities(d3,d1,d2);
            stepMotor(1,pasos3,dutyCycle1);
            stepMotor(2,-pasos1,dutyCycle2);
            stepMotor(3,-pasos2,dutyCycle3);
        }
        else
        {
            checkvelocities(d3,d1,d2);
            stepMotor(1,-pasos3,dutyCycle1);
            stepMotor(2,pasos1,dutyCycle2);
            stepMotor(3,pasos2,dutyCycle3);
        }

        inicio=false;
    }

    if( position <(PI/3.0) || position>(2.0*PI ))
    {
```

```
    rpmobj=0;
    estado=6;

    inicio=true;

    Serial.print("Voy a estado 0, posicion:");
    Serial.println(position);

  }
  break;
}
}
```

Código 8. Máquina de estados.

5.8 BUCLES DE CONTROL

El ESP32, gracias a su arquitectura de doble núcleo, permite ejecutar tareas en paralelo. En el firmware del ESP32 para el control de la longitud de los radios de las ruedas inteligentes, se utilizan dos bucles de control que se ejecutan simultáneamente en diferentes núcleos. Estos bucles gestionan tareas distintas pero complementarias, asegurando un funcionamiento eficiente del sistema. A continuación, se explica cómo se configuran y operan estos bucles.

5.8.1 Bucle 1: Control de Motores Paso a Paso

El primer bucle se encarga de controlar los motores paso a paso mediante una máquina de estados. Este bucle se ejecuta en el núcleo 0.

```
void loop1(void* parameter) {
  for (;;) {

    controlSteps();//Máquina de estados para el control de los radios

  }
}
```

Código 9. Bucle1.

5.8.2 Bucle 2: Manejo de MQTT y PWM

El segundo bucle se encarga de manejar la comunicación MQTT y el control de la PWM (Modulación por Ancho de Pulso) de los motores. Este bucle se ejecuta en el núcleo 1.

En primer lugar, comprueba la conexión con el broker MQTT, realiza el control de la PWM para controlar el consumo de los motores en reposos y por último realiza la lectura de los sensores y envía las lecturas por MQTT.

```
void loop2(void* parameter) { //Comunicación con la Raspberry Pi
    for (;;) {

        //Comprueba la conexión y chequea si hay algún mensaje disponible
        if(!client.connected()){
            reconnect();
        }
        client.loop();

        controlPWM();

        //Lectura de sensores y envío de información por WiFi a la Raspberry Pi
        //Lectura Sensor 1
        float sensor1 = readSensors(1);
        char s1String[8];
        dtostrf(sensor1,1,2,s1String);
        Serial.print("Sensor 1: ");
        Serial.println(s1String);
        if(rueda_R)
        {
            client.publish("esp32/sensor1R",s1String);
        }
        else
        {
            client.publish("esp32/sensor1L",s1String);
        }

        //Lectura Sensor 2
        float sensor2 = readSensors(2);
        char s2String[8];
        dtostrf(sensor2,1,2,s2String);
        Serial.print("Sensor 2: ");
        Serial.println(s2String);
        if(rueda_R)
        {
            client.publish("esp32/sensor2R",s2String);
        }
        else
        {
            client.publish("esp32/sensor2L",s2String);
        }

        //Lectura Sensor 3
        float sensor3 = readSensors(3);
        char s3String[8];
        dtostrf(sensor3,1,2,s3String);
        Serial.print("Sensor 3: ");
        Serial.println(s3String);
        if(rueda_R)
        {
            client.publish("esp32/sensor3R",s3String);
        }
        else
        {
            client.publish("esp32/sensor3L",s3String);
        }
    }
}
```

Código 10. Bucle2.

6 PROGRAMACIÓN DE RASPBERRY PI

En este capítulo, se describe el proceso de configuración y programación de una Raspberry Pi 4 para utilizarse como un broker MQTT (Message Queuing Telemetry Transport). Este broker actúa como intermediario en la comunicación entre los microcontroladores ESP32 que controlan los motores del robot, facilitando la transferencia de datos de manera eficiente y segura.

6.1 INTRODUCCIÓN AL MQTT

MQTT es un protocolo de comunicación ligero diseñado para la transmisión de mensajes en redes con recursos limitados y alta latencia. Es especialmente adecuado para aplicaciones de IoT (Internet de las Cosas) debido a su bajo consumo de ancho de banda y su capacidad para operar de manera confiable en entornos con conectividad intermitente.

Principales características del MQTT:

- **Publicador/Suscriptor:** Los dispositivos se suscriben a tópicos específicos y publican mensajes en estos tópicos.
- **Broker:** Actúa como intermediario para la transmisión de mensajes entre los publicadores y suscriptores.
- **QoS (Quality of Service):** Define tres niveles de entrega de mensajes para asegurar la fiabilidad de la comunicación.

6.2 INSTALACIÓN Y CONFIGURACIÓN DEL BROKER MQTT

La Raspberry Pi 4 se configuró como broker MQTT utilizando el software Mosquitto, que es una implementación de código abierto del protocolo MQTT.

Este se ha configurado con el puerto de escucha 1883 y se quitado el sistema de autenticación para que no sea necesario contraseña para acceder a él.

6.3 PROGRAMACIÓN CON NODE-RED PARA LA GESTIÓN DE MENSAJES

Node-RED es una herramienta de programación basada en flujos para la integración de hardware y servicios de API. Se ha utilizado para gestionar la información transmitida entre los ESP32 y la Raspberry Pi 4.

Para establecer la comunicación entre el Arduino Mega, que controla los motores de corriente continua, y el resto del sistema, se ha utilizado un ESP32. La conexión entre estos dos dispositivos se realiza a través de un puerto serie. El ESP32 recoge la información sobre la velocidad y la posición de las dos ruedas desde el Arduino Mega a través de este puerto serie y luego la transmite mediante el protocolo MQTT.

La configuración de Node-RED está estructurada para recibir datos de las ruedas derecha e izquierda del robot, así como de los sensores de corriente, y procesarlos adecuadamente. Cada flujo consta de nodos de entrada (MQTT In) que reciben los datos desde los ESP32, y nodos de salida (MQTT Out) que reenvían los datos de las posiciones y velocidades de las ruedas.

6.3.1 Conexiones de Velocidad y Posición

El broker recibe los datos de posición y velocidad del ESP32 conectado al Arduino Mega, muestra esta información en el panel de control de Node-Red para que pueda ser monitorizada en tiempo real y la envía a cada una de las ruedas para que esta información sea procesada.

6.3.2 Conexiones de Sensores de Corriente

Cada una de las ruedas envía las mediciones realizadas por cada uno de los tres sensores que tiene cada una. Esta información es recibida por el bróker para que pueda ser monitorizada en tiempo real.

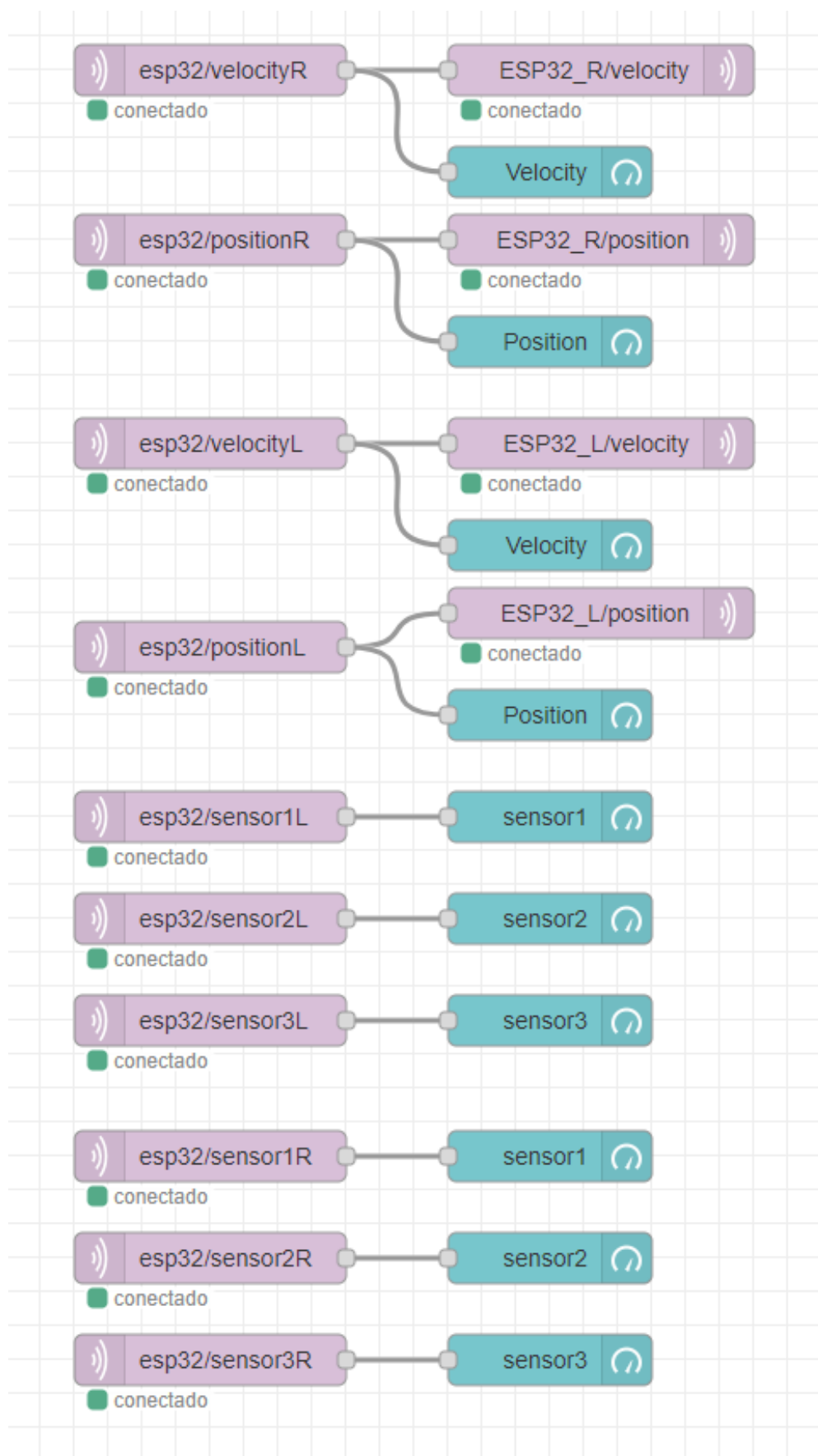


Figura 53. Node-Red.

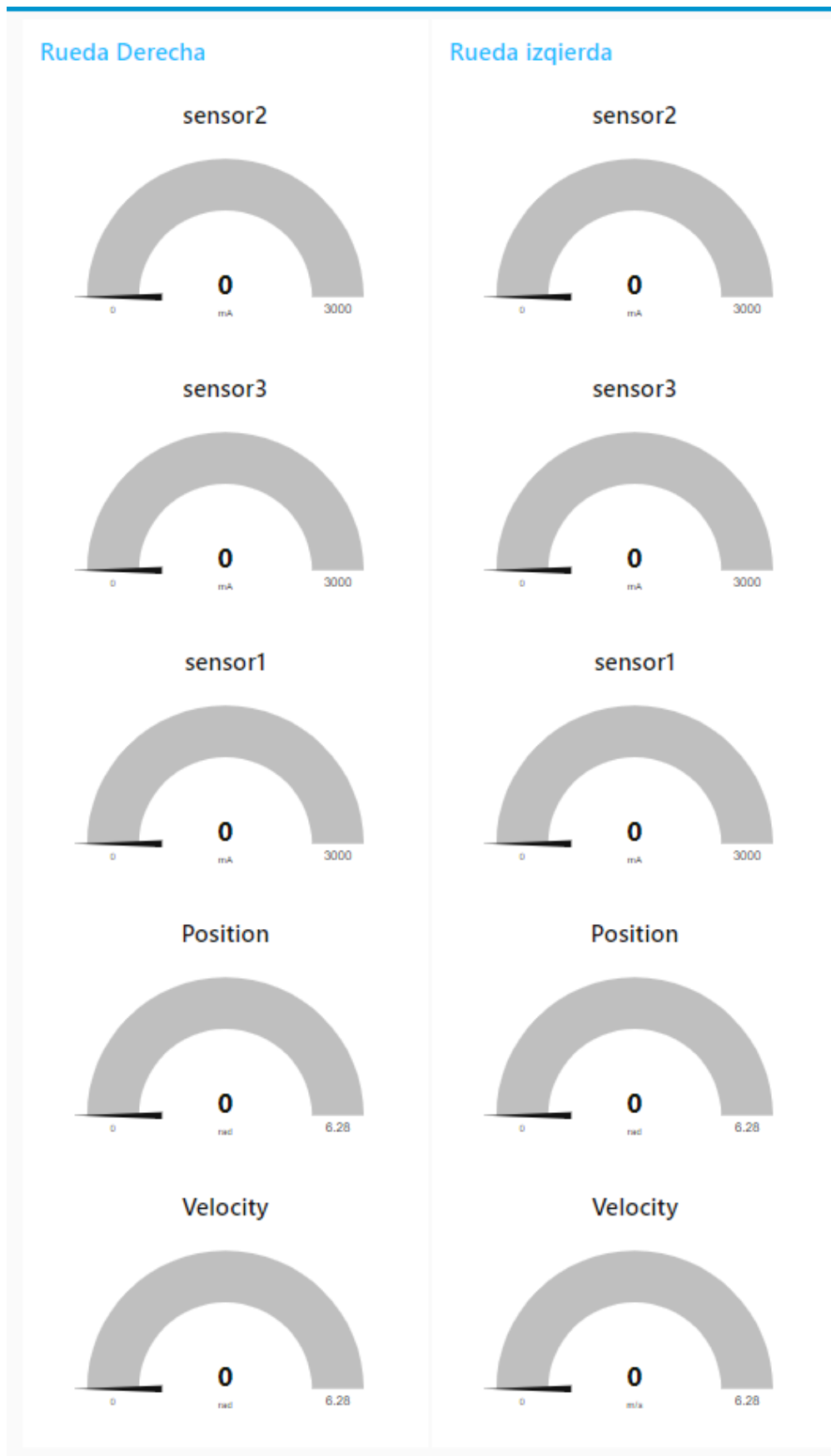


Figura 54. Panel Node-Red.

7 RESULTADOS OBTENIDOS

El desarrollo de este proyecto ha culminado con la creación de una PCB capaz de realizar el control de la rueda inteligente de forma más robusta y eficiente. Algunas de las mejoras han sido:

- **Reducción Considerable del Cableado:** Uno de los logros ha sido la notable reducción del cableado necesario para la comunicación y el control de los motores. Al implementar el microcontrolador ESP32 con capacidades de comunicación Wi-Fi, se ha eliminado la necesidad de una gran cantidad de cables para la conexión de sensores y actuadores, simplificando así el diseño y montaje del sistema.

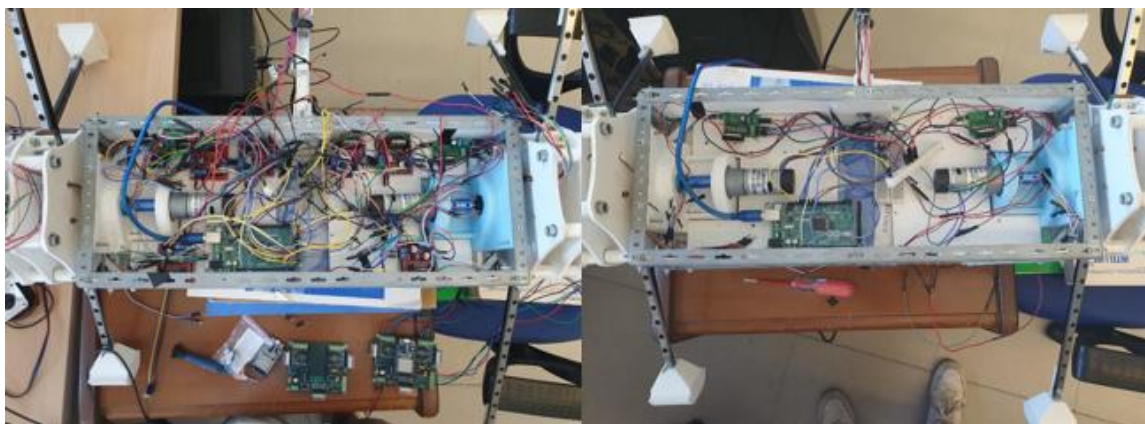


Figura 55. Cableado previo vs actual.

- **Reducción del Consumo Energético:** Otro resultado significativo ha sido la reducción del consumo energético del sistema. Esto ha sido posible gracias a la optimización de la gestión de energía del ESP32 y la implementación de algoritmos eficientes de control de motores que minimizan las pérdidas energéticas. La versión anterior del IMPASS, tenía un consumo de 5 A para los motores paso a paso en cada una de las ruedas, sin embargo, con esta nueva versión se ha obtenido un consumo de 2.5 A para las dos ruedas. En resumen lo que la versión anterior suponía el uso de dos fuentes de alimentación y un consumo de entrono a 10 A se ha transformado en 2.5 A por lo que con una sola fuente de alimentación es más que suficiente.

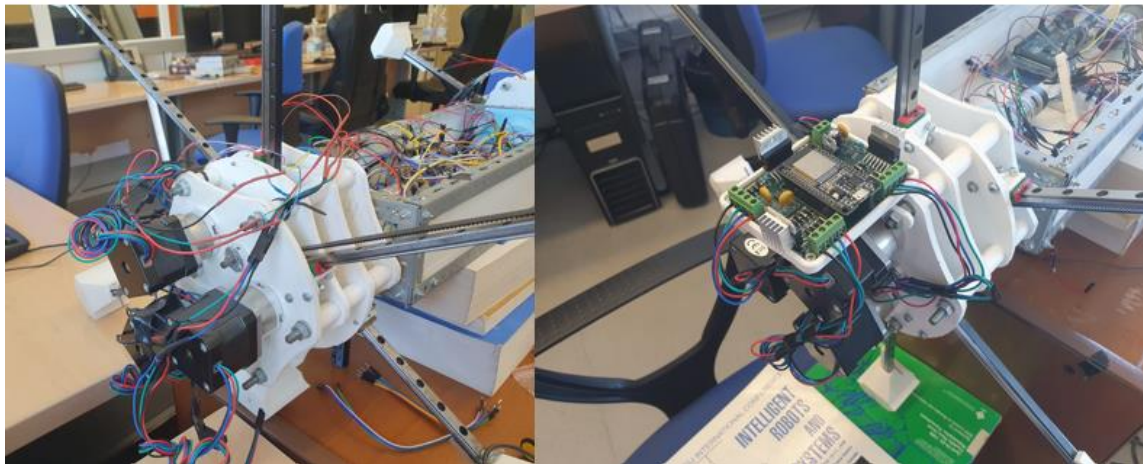


Figura 56. Estado previo vs actual ruedas.

- **Integración de Comunicaciones Inalámbricas:** La capacidad del ESP32 para conectarse a redes Wi-Fi ha facilitado la implementación de un sistema de control remoto y monitoreo en tiempo real. Esto permite ajustar parámetros operativos y supervisar el estado del robot de manera eficiente y en tiempo real desde cualquier dispositivo conectado a la red, mejorando significativamente la flexibilidad y funcionalidad del sistema.
- **Optimización del Espacio y Peso:** Al reducir el cableado y el número de componentes necesarios, también se ha logrado una optimización en el espacio y el peso del sistema. Esto no solo simplifica el montaje y mantenimiento, sino que también da espacio para la implantación de nuevas mejoras como una batería.
- **Implementación de Nuevas Funcionalidades:** Gracias a la flexibilidad del ESP32, se han podido añadir nuevas funcionalidades al sistema, como el monitoreo en tiempo real de la corriente de los motores. Además, se ha integrado la capacidad de enviar y recibir comandos a través de MQTT, facilitando la integración con otros sistemas y plataformas de IoT.

Video del funcionamiento: <https://youtu.be/kEq4EhdIf6U>

Los códigos utilizados para las simulaciones del presente Trabajo de Fin de Grado se encuentran en el siguiente repositorio: <https://github.com/spaceuma/IMPASS-MCU-Wheel>. Este repositorio es privado y para que acceder se debe enviar un email a srl@uma.es

8 CONCLUSIONES Y PUNTOS DE MEJORA

A lo largo del desarrollo de este proyecto se han identificado varias áreas de mejora que podrían incrementar significativamente la autonomía, funcionalidad y eficiencia del robot. A continuación, se detallan algunos de los puntos de mejora propuestos para futuras iteraciones del proyecto.

- **Creación de una PCB para el Control de los Motores de Corriente Continua:** Desarrollar una PCB controlada por un ESP32 para poder realizar la comunicación de todo el sistema por MQTT de forma más simple. Esta debería de esta equipada con un regulador de tensión como la diseñada en este proyecto para poder alimentar todo el sistema con 12V, ya que actualmente es necesario alimentar el Arduino Mega por USB para el funcionamiento del robot. De este modo se reduciría el cableado y se obtendría un sistema más robusto y eficiente.
- **Implementación de una Batería:** A raíz del aumento del espacio disponible gracias a la actual implementación de las PCBs en las ruedas y a una futura PCB para el control de los motores DC que reduciría considerablemente el espacio ocupado, sería interesante la integración de una batería de alta capacidad permitiría al robot operar de manera autónoma sin depender de una fuente de alimentación externa.
- **Implementación de desplazadores electromagnéticos lineales:** El laboratorio está investigando sustituir los motores paso a paso por desplazadores electromagnéticos lineales con un sistema de freno para mantener los radios estáticos. Con esta mejora se pretende reducir los consumos del sistema.
- **Sensores para Conocer la Posición Exacta de Cada Uno de los Radios:** La incorporación de sensores de posición en los radios permitiría conocer su ubicación exacta de los radios en todo momento, eliminando la necesidad de partir siempre de un punto de referencia fijo para el correcto funcionamiento del sistema.
- **Uso de Cámaras para la Implementación de Sistemas de Percepción de Obstáculos:** Integrar cámaras y sistemas de visión por computadora permitiría al robot detectar y evitar obstáculos de manera autónoma. Las cámaras podrían ser utilizadas para implementar algoritmos de percepción visual y mapeo, mejorando significativamente la capacidad del robot para navegar en entornos complejos.

- **Algoritmos para Realizar Mapeo y Localización para Seguimiento de Objetivos:** Implementar algoritmos de mapeo y localización simultánea (SLAM) permitiría al robot construir mapas de su entorno y localizarse dentro de ellos en tiempo real. Esta capacidad es crucial para tareas de exploración y seguimiento de objetivos, permitiendo al robot operar de manera más inteligente y autónoma.

9 REFERENCIAS

Design and Research of All-Terrain Wheel-Legged Robot. (9 de August de 2021).
Obtenido de MDPI Open Access Journals: <https://www.mdpi.com/1424-8220/21/16/5367>

Leo Rover. (s.f.). Obtenido de <https://www.leorover.tech/the-rover>

MDPI. (8 de July de 2022). *A Comprehensive Study of Mobile Robot: History, Developments, Applications, and Future Research Perspectives.* Obtenido de <https://www.mdpi.com/2076-3417/12/14/6951>

NASA. (4 de Dec de 1996). *Mars Pathfinder / Sojourner Rover.* Obtenido de <https://www.jpl.nasa.gov/missions/mars-pathfinder-sojourner-rover>

NASA. (24 de Jan de 2004). *Mars Exploration Rovers: Spirit and Opportunity.* Obtenido de <https://science.nasa.gov/mission/mars-exploration-rovers-spirit-and-opportunity/>

NASA. (5 de August de 2012). *Curiosity Rover.* Obtenido de <https://www.jpl.nasa.gov/missions/mars-science-laboratory-curiosity-rover-msl>

NASA. (s.f.). *Flight Projects.* Obtenido de <https://robotics.jpl.nasa.gov/what-we-do/flight-projects/mars-2020-rover/m2020mobility/>

NASA. (s.f.). *Mars 2020 Perseverance.* Obtenido de https://www.jpl.nasa.gov/news/press_kits/mars_2020/landing/mission/spacecraft/perseverance_rover/

ROMELA. (s.f.). *IMPASS Inteligent Platform with Active Spoke System.* Obtenido de <https://www.romela.org/impass-intelligent-mobility-platform-with-active-spoke-system/>

Stanford University. (16 de January de 2019). *Stanford's robotics legacy.* Obtenido de <https://news.stanford.edu/stories/2019/01/stanfords-robotics-legacy>

The World's First Mobile Intelligent Robot. (1972). Obtenido de Engineering and Technology History Wiki: https://ethw.org/Milestones:SHAKEY:_The_World%E2%80%99s_First_Mobile_Intelligent_Robot,_1972

10 ANEXO A PLANOS PCB

1

2

3

4

A

A

B

B

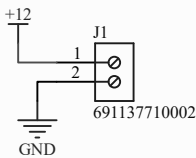
C

C

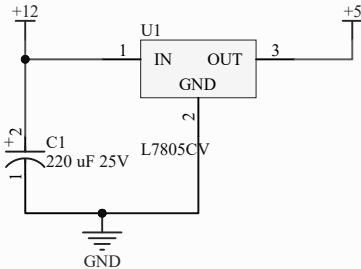
D

D

ENTRADA ALIMENTACIÓN 12V



REGULADOR DE TENSIÓN



Title			
Alimentación y Regulador de Tensión			
Size	Number	Revision	
A4	Alimentación y Controlador de Tensión		
Date:	6/29/2024	Sheet of	
File:	C:\Users\...\Supply.SchDoc	Drawn By:	Alimentación y Controlador de Tensión

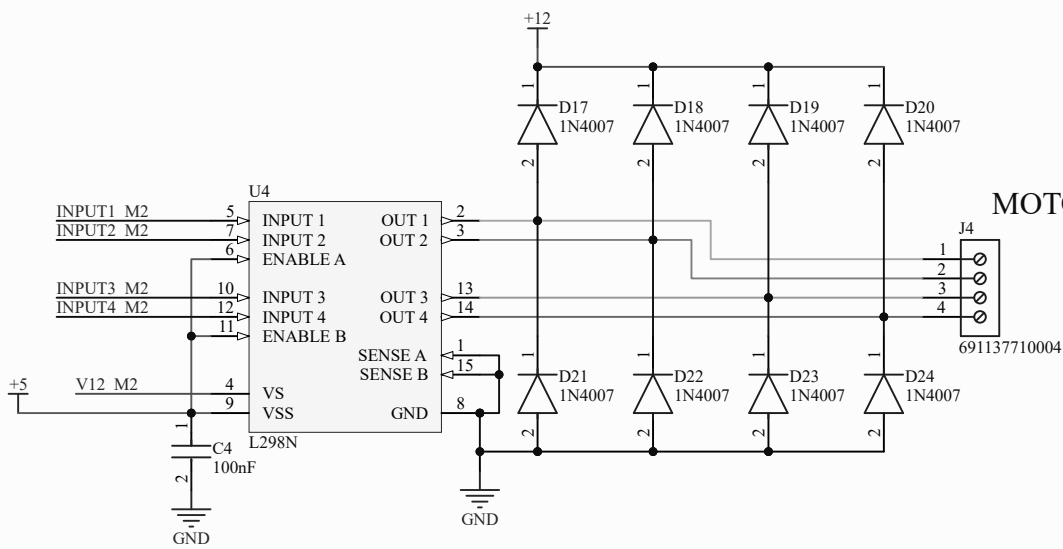
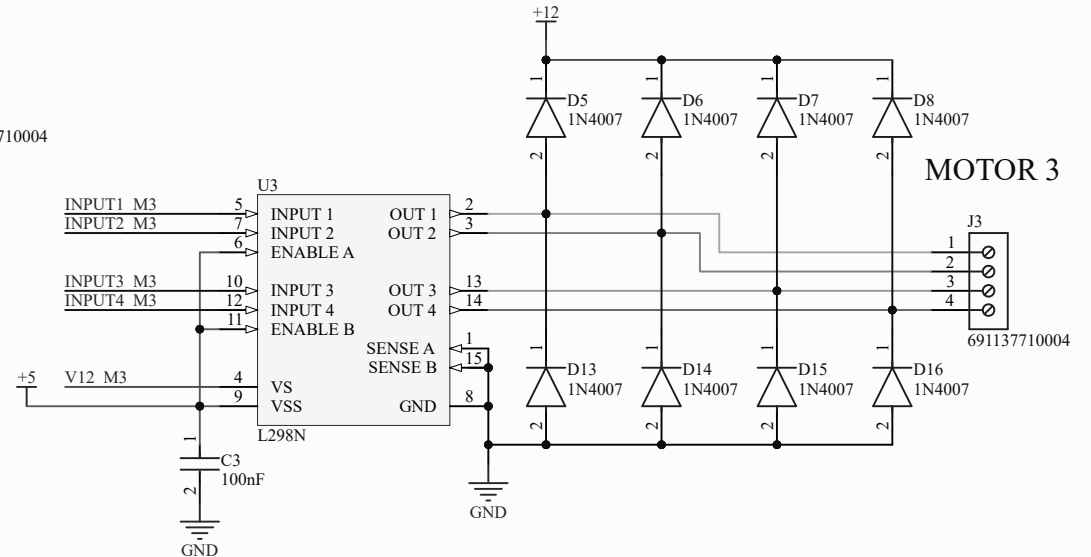
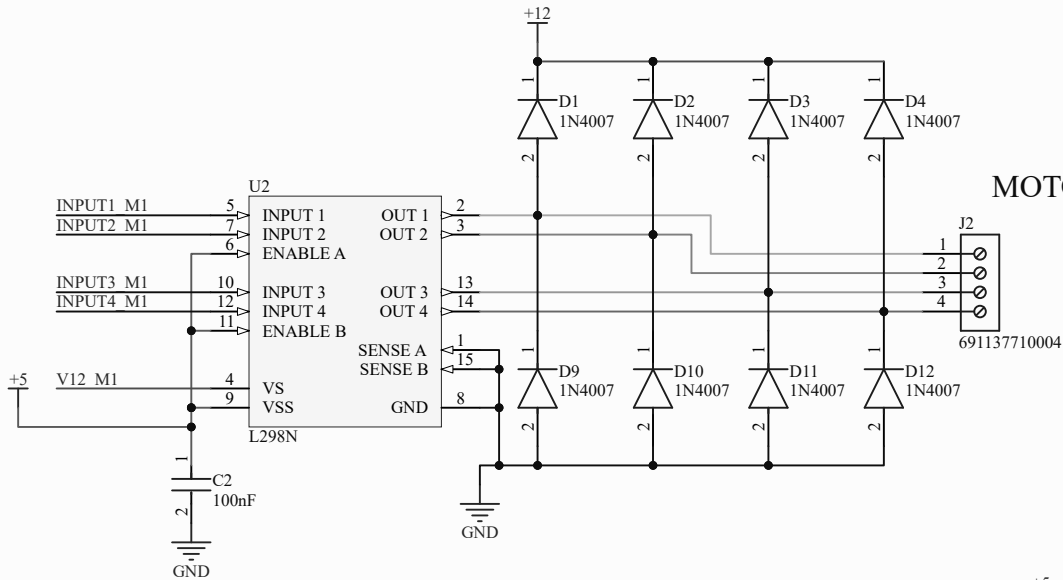
1

2

3

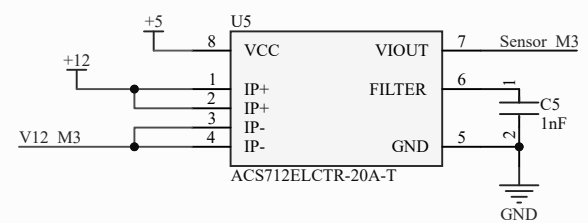
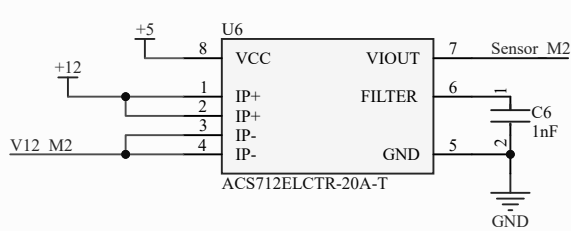
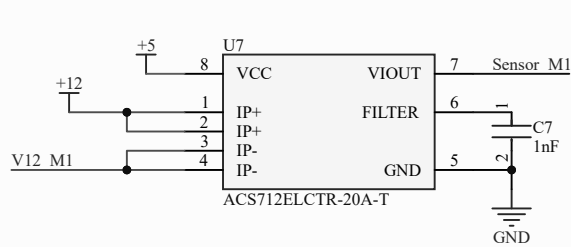
4

CONTROLADORES MOTORES PASO A PASO



Title		
Controladores Motores Paso a Paso		
Size	Number	Revision
A4	2	
Date:	6/29/2024	Sheet of
File:	C:\Users\...\Controllers.SchDoc	Drawn By: José María Hermoso Ocaña

SENSORES DE CORRIENTE



Title Sensores de Corriente			
Size A4	Number 3		Revision
Date:	6/29/2024	Sheet of	
File:	C:\Users\...\Sensors.SchDoc	Drawn By:	José María Hermoso Ocaña

